ARMOR-FL: Anytime-Valid Robust Martingale-Based Online Reweighting for Federated Intrusion Detection

Quang-Vinh Dang[1*]

[1*]British University Vietnam, Hung Yen, Vietnam.

Corresponding author(s). E-mail(s): vinh.dq4@buv.edu.vn;

Abstract

Federated intrusion detection lets network operators train a shared classifier without pooling raw traffic, but it opens the aggregator to poisoned client updates. Existing Byzantine-robust defenses compare each round's updates against the population and flag outliers, yet an anytime-valid comparison of this kind is mathematically guaranteed to eventually reject any client whose behavior sits persistently off-center – so it cannot tell a merely non-IID honest client from an attacker without further information. We diagnose this failure concretely: a population-only e-process excluded up to 62.5% of honest clients under IID data in our experiments, the opposite of what intuition suggests. We introduce ARMOR-FL, which adds a self-referential shift test – has this client's own deviation changed, not just how large is it – alongside a scale-robust population statistic and a threshold-calibrated trust weight, to recover the sequential-testing guarantee without this failure mode. Across CICIDS2017, CICIDS2018, and CICIoT2023 under label-flipping, Gaussian-noise, and adaptive (ALIE) attacks, ARMOR-FL removes the false-exclusion pathology entirely and outperforms undefended averaging, though a distance-based Krum baseline remains stronger in aggregate; we report this honestly, including the specific conditions – non-IID data under lighter poisoning (a moderate 20% malicious client fraction) – where ARMOR-FL's mechanism gives it a real edge over Krum. We release the full implementation, configuration grids, and reassembly-only dataset archives.

Keywords: Federated learning, Byzantine robustness, intrusion detection, anytime-valid inference, sequential testing, non-IID data

1 Introduction

A network intrusion detection system trained on one organization's traffic sees only that organization's attacks. Pooling traffic logs across organizations would give any classifier a richer view of what an attack looks like, but the logs themselves are exactly the kind of data no organization wants to hand over: source and destination addresses, timing, payload sizes, and often enough metadata to reconstruct who was talking to whom. Federated learning offers a way around this tension. Each site trains locally and shares only model updates, and a central aggregator combines them into a global classifier that none of the participants could have trained alone [1].

That convenience creates a new attack surface. An aggregator that averages every client's update by default has no way to tell a legitimate update from one designed to corrupt the global model – and unlike a centrally-trained model, an attacker in this setting does not need access to the training data at all, only membership in the federation. A single compromised or malicious

participant can flip labels, inject noise into its local training, or craft updates specifically to survive whatever averaging rule the aggregator uses. This is the Byzantine-robustness problem in federated learning, and it is particularly consequential for intrusion detection: an attacker with the ability to degrade the very system meant to detect it has an unusually strong incentive to try.

The dominant family of defenses addresses this by treating each round's updates as a small sample and testing whether any one of them deviates enough from the rest to be flagged as an outlier. Krum scores each client by distance to its nearest neighbors and keeps only the most central update [2]; FoolsGold tracks the cosine similarity of a client's updates over time to catch colluding attackers who behave too similarly to one another [3]; robust-statistics baselines such as trimmed mean and coordinate-wise median discard the extremes of the per-coordinate distribution before averaging. What these methods share, whatever their specific mechanism, is that they compare a client against the *population* in the current round – and the population, in a real deployment, is rarely identically distributed. Federated intrusion detection is a canonical non-IID setting: different sites see different network topologies, different device populations, and different background traffic, so it would be surprising if their honest updates ever looked interchangeable.

Our starting point for this paper was a puzzle that surfaced while building a defense meant to solve exactly this. Anytime-valid sequential testing – the machinery behind e-processes and testing-by-betting [4] – lets a defender accumulate evidence about a client across many federated rounds rather than making an isolated decision each round, and gives a formal guarantee (via Ville's inequality) on the false-exclusion rate no matter when the defender chooses to look. This sounded like the right tool for exactly the population-comparison problem above: instead of a single noisy per-round distance, accumulate evidence over time and only act once the evidence crosses a principled threshold. We built this, called it ARMOR-FL, and then found, while testing it against synthetic honest clients with no attacker present at all, that it was excluding them anyway – sometimes catastrophically so.

The reason turned out to be structural rather than a bug in our implementation. An anytime-valid test of the form "is this client's distance from the population median elevated" is, by the very guarantee that makes it anytime-valid, mathematically certain to eventually reject any client whose true distance sits persistently above that median – given enough rounds, the martingale accumulates enough evidence regardless of whether the elevation reflects an attack or simply a client whose local data looks different from everyone else's. Longer runs do not average this out; they make the eventual rejection more certain. A defense built this way does not fail occasionally under non-IID data. It is guaranteed to fail, given enough time, for any client that never converges toward the population's center – which describes an ordinary heterogeneous participant just as well as it describes an attacker with a fixed, persistent bias.

We confirmed this is not an edge case. In a controlled test with ten honest clients and zero attackers, five clients configured to be moderately more variable than the rest (a stand-in for ordinary non-IID heterogeneity) were excluded within twenty rounds by a population-only e-process, at a false-exclusion rate the design's stated significance level does not come close to explaining. On real network traffic the effect was worse: under CICIDS2017 with a purely IID partition and a 20% malicious client fraction – the setting we expected to be *easiest* for a population comparison, since IID data has no legitimate heterogeneity to confuse with attack – the uncorrected defense excluded 62.5% of honest clients. The population becoming more homogeneous made the statistic *more* fragile, not less: with less genuine spread in the honest updates, the population's own median-absolute-deviation shrinks, and dividing ordinary training noise by a near-zero scale estimate manufactures large deviation scores from clients that are not doing anything unusual at all.

This paper reports the resulting design, ARMOR-FL, together with the diagnostic process that produced it, because we think the process is as informative as the fix. ARMOR-FL keeps the population e-process but adds a second, self-referential test: has this client's own deviation *changed* relative to its own recent history, as opposed to how large the deviation currently is. A client that has always looked somewhat off-center produces no shift signal and is protected

from exclusion; a client whose deviation appears mid-run, coinciding with when an attack plausibly begins, does produce one, and only the combination of population-level and self-referential evidence triggers hard exclusion. We pair this with two further corrections found in the course of validating the fix: a population scale estimate that cannot collapse below a fraction of its own recent history, closing the IID fragility described above, and a trust-weighting rule recalibrated against the same threshold that governs exclusion, so that ordinary deviation is not punished at a rate disproportionate to the actual evidence against a client.

We are explicit that this is not a paper reporting a method that wins on every axis. Across three widely used intrusion-detection benchmarks – CICIDS2017, CICIDS2018, and CICIoT2023 – and three attack types of increasing subtlety, ARMOR-FL removes the false-exclusion pathology it was built to fix and clearly outperforms undefended federated averaging, but Krum remains the stronger baseline in aggregate on all three datasets. What ARMOR-FL does deliver is a specific, identifiable regime – non-IID client populations under lighter label-flipping poisoning – where its mechanism gives it a genuine edge, together with formal anytime-valid guarantees that purely empirical defenses such as Krum and FoolsGold do not provide, and an honest account of where it still falls short. We think that account, including the parts where our own method loses, is a more useful contribution to this literature than a narrower set of results engineered to look uniformly favorable.

Our contributions are threefold. First, we identify and formally characterize a failure mode shared by population-only Byzantine-robust defenses in non-IID federated learning: any anytime-valid test of population deviation alone is guaranteed to eventually exclude persistently off-center honest clients, and this failure is most acute, not least acute, under low heterogeneity, where the population's own scale estimate becomes fragile. Second, we introduce ARMOR-FL, a concrete fix combining a self-referential shift test, a scale-floored population statistic, and a threshold-calibrated trust weight, and we validate each component both on isolated synthetic reproductions of the failure it targets and on three real intrusion-detection datasets. Third, we provide, to our knowledge, the first reported robustness evaluation of a Byzantine-robust federated defense on CICIoT2023 under an explicit poisoning-attack grid, alongside a full open implementation, configuration files, and reassembly-only dataset archives so the entire grid can be reproduced without re-downloading from the original hosts.

The remainder of the paper is organized as follows. Section 2 situates ARMOR-FL against Byzantine-robust federated learning, non-IID drift handling, and sequential testing, and identifies the specific gap our diagnostic process fills. Section 3 describes the backbone intrusion-detection model and the ARMOR-FL aggregation algorithm, including the three design decisions motivated by the failure modes above. Section 4 describes the datasets, attacks, and experimental grid. Section 5 reports results, including a detailed case study of the one setting where ARMOR-FL's mechanism visibly earns its keep. Section 6 discusses when and why the method helps or does not, and Section 8 states the limitations we did not resolve, before Section 9 concludes.

2 Related Work

2.1 Byzantine-robust aggregation in federated learning

The starting point for distance-based Byzantine robustness is Krum [2], which scores each client's update by the sum of squared distances to its nearest neighbors and selects the most central one as the round's sole contribution, discarding the rest outright. Multi-Krum generalizes this to a small committee of central updates rather than one, and Bulyan [5] layers a coordinate-wise trimmed-mean filter on top of a Multi-Krum-style selection specifically to close a residual vulnerability – a small number of colluding attackers crafting updates that individually pass Krum's distance check but jointly bias a specific coordinate – that Krum alone does not address. Coordinate-wise trimmed mean and coordinate median trade Krum's winner-take-all selection for a per-coordinate robust average, tolerating a bounded fraction of adversarial coordinates without needing to identify which clients contributed them. FoolsGold instead exploits a specific attacker behavior – Sybil clients whose updates correlate with each other over time –

and downweights clients whose update history is too similar to another's [3]. Divide-and-Conquer (DnC) [6] was introduced specifically to counter adaptive, omniscient attacks in the ALIE family via a spectral-projection filter rather than a distance-based one, and is a natural point of comparison for our own ALIE results (Section 5) that we do not evaluate directly, since our contribution is the anytime-valid sequential-testing layer rather than a new per-round filter. Karimireddy, He, and Jaggi [7] explicitly diagnose a closely related tension – that standard Byzantine-robust aggregation rules lose their theoretical guarantees under client heterogeneity because heterogeneity inflates the same cross-sectional variance the aggregation rule relies on to detect attackers – and propose worker momentum as a heterogeneity-robust fix; their diagnosis targets the aggregation rule's convergence guarantee under heterogeneity in general, whereas ours targets a different, specific failure of anytime-valid *testing* rules applied on top of any such aggregation rule, and the two are complementary rather than overlapping contributions. All of these methods make a decision about the current round using only the current round's cross-sectional spread of updates, which is exactly the property that makes them vulnerable to conflating genuine non-IID heterogeneity with an attack: a client that is consistently, but honestly, different from the rest of the population looks identical, from a single round's perspective, to one that is consistently malicious.

More recent work in this space targets federated intrusion detection specifically. Dependable federated learning against poisoning attacks [8] combines loss-scoring with Manhattan-similarity clustering to recover accuracy after a label-flipping attack on CIC-IDS-2017. Lavaur et al. [9] run a systematic study of label-flipping specifically, reporting that even a well-tuned FL-IDS pipeline can lose over ten points of accuracy under attack, a finding consistent with what we observe for undefended averaging in our own experiments. GRAF-IDS [10] and the related federated-RNN defense [11] build graph-based clustering aggregation with Krum-style selection layered on top, evaluated on IoT-focused datasets rather than the CIC family. WeiDetect [12] models the per-client deviation distribution with a Weibull fit rather than the empirical median-absolute-deviation approach common elsewhere, which is conceptually adjacent to our own use of a robust scale statistic, though it does not incorporate any sequential or anytime-valid guarantee. D3-FLIDS [13] pairs gradient projection with isolation-forest scoring; FedDBC [14] targets density-based collusion detection specifically, one of the few defenses in this literature that explicitly addresses coordinated multi-cluster attacks rather than independent ones. FLEX-IDS [15] adds explainability on top of a federated defense evaluated at up to 30% malicious participation, the same upper bound we use in our own malicious-fraction grid.

Beyond this core set, the broader federated-intrusion-detection literature published in *Cluster Computing* itself gives a sense of how active this specific venue is on adjacent problems: a fair-client-selection framework combining earth-mover's-distance-based selection, fully homomorphic encryption, and SMOTE-based rebalancing [16]; a recent survey specifically on federated learning for intrusion detection [17]; a lightweight-LLM hierarchical federated scheme for B5G-enabled IoT intrusion detection [18]; a federated SimpleNet-based defense for cyber-physical systems [19]; a federated deep-Q-learning approach to IoT security [20]; and a trust-aware framework for privacy-preserving smart-city intrusion detection [21] together with a blockchain-assisted trust framework for software-defined networks [22]. None of these six papers combines Byzantine-robust aggregation with an anytime-valid statistical guarantee, which is where ARMOR-FL's contribution sits relative to this venue's existing coverage.

Two papers deserve closer differentiation because they are the closest in spirit to ARMOR-FL's own contribution. Our own prior work, FORTRESS-FL [23], is a Byzantine-robust and privacy-preserving federated orchestration framework for next-generation networks; it shares the goal of robust aggregation under adversarial participants but has no anytime-valid statistical component and no mechanism to decouple drift from attack, which is precisely the gap ARMOR-FL is built to close. RPCFL [24], published in this journal, is a Byzantine-robust and privacy-preserving clustered federated learning framework built on secure multiparty computation and dynamic clustering; it is, to our knowledge, the only other Byzantine-robust federated learning paper published in *Cluster Computing* itself. RPCFL's

robustness guarantee comes from cryptographic clustering rather than statistical testing, and it does not claim or provide the anytime-valid false-exclusion control that is ARMOR-FL's central methodological contribution; the two approaches are complementary rather than competing, and a system combining SMPC-based privacy with anytime-valid robustness testing is a natural direction for future work.

2.2 Qualitative comparison of defense properties

Table 1 summarizes the eight defenses surveyed above (plus ARMOR-FL) along four properties that this paper's contribution turns on: whether the defense provides a formal, cumulative statistical guarantee rather than a per-round heuristic; whether it has any mechanism to decouple stable, honest heterogeneity from an evolving attack signal; its dominant per-round computational cost in the notation of Section 3.5; and whether it specifically targets coordinated (Sybil-style) attackers as opposed to independent ones.

Three observations follow from this comparison. First, ARMOR-FL is, to our knowledge, the only defense in this table offering a formal cumulative false-exclusion guarantee, but this comes with the practical cost documented throughout this paper: the guarantee is only as good as the population statistic it is built on, and Section 5 shows it does not translate into an aggregate accuracy advantage over Krum's purely empirical rule. Second, we distinguish two different senses of "decoupling heterogeneity from attack." Karimireddy et al. [7] address a real and closely related problem – that standard robust-aggregation rules lose their convergence guarantee under client heterogeneity, because heterogeneity inflates the same cross-sectional variance the rule uses to detect attackers – and fix it via worker-momentum smoothing applied to the aggregation rule itself. ARMOR-FL addresses a different failure, specific to *sequential exclusion decisions*: whether a test that accumulates evidence across rounds can tell a client whose deviation is persistent-but-stable from one whose deviation is attack-coincident. No other defense in this table (including Karimireddy et al.'s momentum-based fix, which smooths the aggregation rule rather than gating an exclusion decision) provides this second, sequential-testing form of decoupling, which is the specific gap our claim in Section 2.2 refers to – the two mechanisms are complementary rather than competing, and momentum smoothing inside ARMOR-FL's own local training step is a plausible additional layer we have not tested. Third, RPCFL's guarantee is cryptographic (secure-multiparty-computation-based correctness) rather than statistical, making it complementary to, rather than a substitute for, ARMOR-FL's approach – a system combining both remains, as noted above, a natural direction for future work.

2.3 Drift, heterogeneity, and the limits of population comparison

The tension between non-IID heterogeneity and outlier-based robustness has been noted before, but rarely diagnosed at the level of a specific, quantified failure mode. Our own earlier work on spatiotemporal federated explainable intrusion detection, ST-FedXIDS [25], addresses drift adaptation via graph learning for high-density WiFi environments, but treats drift as something to adapt to rather than something to formally decouple from an ongoing statistical test of attack; we cite it here for the drift-adaptation half of the present contribution only, since ST-FedXIDS does not itself provide a population-versus-attack decoupling mechanism. Uncertainty-measure approaches to intrusion detection [26] motivate the general statistical-testing framing we build on, without addressing the federated non-IID setting directly.

The broader distributed-systems literature on non-IID federated learning acknowledges that client heterogeneity degrades the accuracy of naive robust-aggregation rules, but the standard response is architectural – personalization layers, clustering clients by similarity before aggregating within clusters, client-side regularization to keep local updates closer to the global model, or, as in Karimireddy et al.'s worker-momentum approach [7], smoothing the aggregation rule itself so its convergence guarantee survives heterogeneous inputs – rather than a change to the statistical test a Byzantine defense uses to decide whether a client is suspicious in the first place. None of these architectural responses touches the sequential-exclusion mechanism specifically, which

Table 1 Qualitative comparison of Byzantine-robust federated defenses

Defense	Statistical guarantee	Drift/attack decoupling	Dominant cost	Sybil-targeted
FedAvg (undefended)	None	N/A	$O(Kd)$	No
Krum [2]	None (empirical)	None	$O(K^2d)$	No
Bulyan [5]	None (empirical)	None	$O(K^2d)$	No
Trimmed mean / coord. median	None (empirical)	None	$O(Kd \log K)$	No
FoolsGold [3]	None (empirical)	None	$O(K^2)$	Yes
DnC [6]	None (empirical)	None	$O(Kd)$ (spectral)	No
Karimireddy et al. [7]	None (empirical)	Worker-momentum smoothing	$O(Kd)$	No
FORTRESS-FL [23]	None (empirical)	None	Not reported	No
RPCFL [24]	None (cryptographic)	None	SMPC-dependent	No
ARMOR-FL (ours)	Anytime-valid (α-controlled)	Self-shift e-process	$O(Kd)$	No

is where our diagnosis is targeted: to our knowledge, no prior published defense identifies the specific mechanism we diagnose here: that an anytime-valid population test is not merely inaccurate under heterogeneity but is mathematically *guaranteed* to eventually misclassify a persistently off-center honest client, with the failure growing more severe, not less, as the population becomes more homogeneous. This last point in particular runs against the intuitive expectation that IID data should be the easy case for any population-comparison method, and we are not aware of it being reported elsewhere in this form.

2.4 Anytime-valid inference and its (limited) use in federated defenses

E-processes and testing-by-betting [4] generalize classical sequential testing to give a valid Type-I error guarantee at every stopping time, not just at a pre-registered sample size, via Ville's maximal inequality for nonnegative supermartingales. Ramdas, Grünwald, Vovk, and Shafer's survey of game-theoretic statistics and safe anytime-valid inference [27] situates this specific construction within a broader family of time-uniform guarantees, including Howard, Ramdas, McAuliffe, and Sekhon's time-uniform, nonasymptotic confidence sequences [28], which generalize the same nonnegative-supermartingale machinery to interval estimation rather than binary hypothesis rejection; both are foundational to the framework we build on and neither has, to our knowledge, been previously applied to the Byzantine-exclusion setting this paper addresses. This body of theory has seen substantial recent uptake in statistics and machine learning more broadly for problems requiring continuous monitoring – clinical trial interim analysis, A/B testing, and drift detection – but its adoption inside federated-learning Byzantine defenses specifically has been limited. Most robust-aggregation baselines, including all of those surveyed above, make an independent per-round decision with no formal guarantee on the cumulative false-exclusion probability across a training run of arbitrary length. ARMOR-FL's use of a fixed, theoretically grounded null mean (rather than online calibration from a client's own early-round behavior, which would let an already-compromised client calibrate itself as normal) is, as far as we are aware, a design choice not previously discussed in this literature, and one we believe is necessary rather than merely convenient: any online-calibrated null risks exactly the self-normalization failure it is meant to prevent.

2.5 Dataset coverage

CICIDS2017 [29] and its successor CSE-CIC-IDS2018 remain the most widely used intrusion-detection benchmarks in the Byzantine-robust federated learning literature surveyed above; every defense we discuss in Section 2.1 reports results on one or both. CICIoT2023 [30], a larger and more recent IoT-focused traffic dataset, has seen substantial adoption for centralized intrusion detection [31, 32] but, to our knowledge, no prior Byzantine-robust federated defense with an explicit poisoning-attack evaluation has reported results on it. This is a first-mover contribution of the present work: the fact that ARMOR-FL's aggregate performance on CICIoT2023 is markedly weaker across all four aggregators we test (Table 3 in Section 5) than on the two CIC datasets is itself new information for this literature, and one we discuss further in Section 6.

The present work also builds directly on FedSE-1DSqueezeNet [33], published in this journal, whose SE-1DSqueezeNet backbone architecture we reimplement and reuse without modification (Section 4); our contribution is entirely at the aggregation layer, and we adopt FedSE-1DSqueezeNet's own federated hyperparameters (Table 2) as closely as computational constraints allow, disclosed explicitly in Section 4.

Beyond the federated setting, recent centralized intrusion detection work on these same datasets gives a sense of the classification ceiling each dataset admits absent any federated partitioning or poisoning at all: few-shot cross-domain detection on CICIDS2017/2018 [34], transformer-and-CNN-BiLSTM hybrids [35], transformer-versus-CNN-LSTM comparisons on CSE-CIC-IDS2018 [36], an autoencoder-hybrid-LSTM-CNN residual network [37], a multiscale-attention 1D-CNN approach on large-scale IoT traffic [38], and GAN-based data augmentation for CICIDS2017 [39]. None of these centralized methods addresses the federated poisoning-robustness question this paper is concerned with, but they establish that the underlying classification task is itself solvable to a high standard when data can be pooled centrally, which is useful context for interpreting the accuracy ceilings in Section 5.

Our own prior work spans IoT malware detection via federated learning [40], conformal-prediction-based botnet detection [41], cross-dataset transfer for intrusion detection [42], general IoT intrusion detection [43], foundational machine-learning-for-intrusion-detection work [44], and explainability-driven intrusion detection [45]; none combines Byzantine robustness with anytime-valid inference, which is the specific gap the present work fills. A separate personalized federated-learning poisoning defense not authored by us [46] similarly stops short of an anytime-valid statistical guarantee.

3 Method

3.1 Preliminaries: anytime-valid sequential testing

Before describing ARMOR-FL's aggregation pipeline, we make precise the statistical machinery underlying it, since the failure mode motivating this paper (Section 1) is a consequence of this machinery's own guarantee rather than an implementation defect, and stating it formally lets us prove that claim rather than merely assert it.

Definition 1 (e-process) Let $(\mathcal{F}_t)_{t\geq 0}$ be a filtration representing the information available to the aggregator after round t. A process $(E_t)_{t\geq 0}$ adapted to $(\mathcal{F}_t)$ is an *e-process* for a null hypothesis H_0 if $E_0 = 1$ and (E_t) is a nonnegative supermartingale under H_0, i.e. $E_t \geq 0$ and $\mathbb{E}[E_t \mid \mathcal{F}_{t-1}] \leq E_{t-1}$ for all $t \geq 1$.

Theorem 1 (Ville's inequality, anytime-valid Type-I control) *Let (E_t) be a nonnegative supermartingale with $E_0 = 1$ under H_0. Then for any $\alpha \in (0, 1)$,*

$$\Pr_{H_0}[\exists\, t \geq 0 : E_t \geq 1/\alpha] \leq \alpha.$$

Theorem 1 is the classical result underlying testing-by-betting [4]; we restate it here because it is the exact guarantee ARMOR-FL's population e-process relies on, and because its proof (a direct consequence of the supermartingale maximal inequality) makes clear that the guarantee holds uniformly over *every* stopping rule, including one that stops the first time E_t crosses $1/\alpha$ – which is precisely how the aggregator decides when to exclude a client. This is the entire

appeal of the anytime-valid framework over classical fixed-sample testing: the defender does not need to commit in advance to how many rounds of evidence to collect before deciding, and the false-exclusion probability is controlled at level α no matter when, or how adaptively, that decision is made.

ARMOR-FL instantiates (E_t) via the testing-by-betting construction: given a per-round statistic $z_k^{(t)}$ for client k with null mean $\mathbb{E}[z_k^{(t)} \mid \mathcal{F}_{t-1}] = \mu_0$ under H_0 (honest behavior), the per-round multiplier is $e_t = 1 + \lambda(z_k^{(t)} - \mu_0)$ for a fixed betting fraction $\lambda \in (0,1)$, and $E_t = \prod_{s \le t} e_s$. Because $\mathbb{E}[e_t \mid \mathcal{F}_{t-1}] = 1$ under H_0, (E_t) is exactly a nonnegative martingale (hence supermartingale) with $E_0 = 1$, so Theorem 1 applies directly. ARMOR-FL fixes $\mu_0 = 1$ for the population statistic (Section 3.2) rather than calibrating it online from each client's own early behavior, since online calibration would let an already-compromised client's early malicious rounds define its own null, defeating the guarantee entirely.

Proposition 2 (Guaranteed eventual exclusion under persistent elevation) *Suppose a client's true per-round statistic satisfies $\mathbb{E}[z_k^{(t)} \mid \mathcal{F}_{t-1}] = \mu > 1$ for every round t (a client whose deviation from the population center is persistently, not merely transiently, elevated), and suppose in addition that $\mathrm{Var}(z_k^{(t)} \mid \mathcal{F}_{t-1}) \le \sigma^2 < \infty$ uniformly in t – a mild regularity condition satisfied whenever the client's round-to-round training dynamics are stable rather than diverging, which we take as given throughout (both honest heterogeneous clients and the attacks in our experimental grid produce stable, non-diverging per-round distances). Then for a suitable fixed betting fraction $\lambda \in (0,1)$, $\log E_t \to +\infty$ almost surely as $t \to \infty$, so for any significance level $\alpha \in (0,1)$ there exists an almost-surely finite round T at which $E_T \ge 1/\alpha$, i.e. the client is eventually excluded with probability one.*

Proof sketch Let $f(\lambda) = \mathbb{E}[\log(1 + \lambda(z_k^{(t)} - 1))]$ under the true (non-null) distribution of $z_k^{(t)}$, evaluated at the persistent mean μ. We have $f(0) = 0$ and, by dominated convergence, $f'(0) = \mathbb{E}[z_k^{(t)} - 1] = \mu - 1 > 0$. Hence there exists $\lambda^* \in (0,1)$ small enough that $f(\lambda^*) > 0$. Writing $X_t = \log e_t = \log(1 + \lambda^*(z_k^{(t)} - 1))$, note that $z_k^{(t)} \ge 0$ and $\lambda^* < 1$ keep the argument $1 + \lambda^*(z_k^{(t)} - 1)$ bounded below by $1 - \lambda^* > 0$ (though not bounded above, since $z_k^{(t)}$ itself need not be), so the derivative of $\log(1 + \lambda^*(z-1))$ with respect to z is bounded above by $\lambda^*/(1-\lambda^*)$ for every $z \ge 0$; a mean-value bound then gives $\mathrm{Var}(X_t \mid \mathcal{F}_{t-1}) \le (\lambda^*/(1-\lambda^*))^2 \sigma^2$ uniformly in t, using only the assumed uniform variance bound on $z_k^{(t)}$ rather than any bound on $z_k^{(t)}$ itself. With $\sum_t \mathrm{Var}(X_t \mid \mathcal{F}_{t-1})/t^2 < \infty$ satisfied by this uniform bound, Kolmogorov's strong law for martingale difference sequences gives $\frac{1}{t}\sum_{s \le t}\big(X_s - \mathbb{E}[X_s \mid \mathcal{F}_{s-1}]\big) \to 0$ almost surely, and since $\mathbb{E}[X_s \mid \mathcal{F}_{s-1}] = f(\lambda^*) > 0$ for every s under the persistent-mean hypothesis, $\frac{1}{t}\sum_{s \le t} X_s \to f(\lambda^*) > 0$ almost surely. Therefore $\log E_t = \sum_{s \le t} X_s \to +\infty$ almost surely, and since $\log(1/\alpha)$ is a fixed finite threshold, the sequence crosses it in finite time almost surely. □

We are explicit that Proposition 2 is, at its mathematical core, a restatement of a well-known property of sequential tests – that a test's power against a fixed persistent alternative tends to one as the number of observations grows, a consequence traceable to Wald's original sequential probability ratio test [47] and standard in the testing-by-betting literature we build on [4, 27]. What we contribute is not this consistency property itself but the observation that, applied to a population-comparison statistic as a Byzantine-exclusion gate, consistency is precisely the failure mode: nothing in the proposition's hypotheses distinguishes an attacker from a merely heterogeneous honest client. Both produce $\mu > 1$ persistently whenever their true update distribution sits off-center relative to the trimmed-mean reference point, and Proposition 2 guarantees both are eventually excluded by a population-only e-process with probability one, given enough rounds. This is precisely the mechanism responsible for the false exclusions we report in Section 1 and reproduce synthetically in Section 3.3: it is not that the test is miscalibrated or under-powered, but that its power against *any* persistent alternative – honest or malicious – is, by design, eventually one. A defense built purely on Theorem 1 therefore requires an additional signal that distinguishes *why* $\mu > 1$, which is exactly the role of ARMOR-FL's self-shift test (Section 3.3); the genuinely new contribution is this bridging diagnosis and its fix, not the underlying consistency mathematics.

Remark 1 (Scale-estimator fragility under low heterogeneity) A related but distinct fragility affects the

scale estimate $\mathrm{MAD}^{(t)}$ itself rather than the e-process built on top of it. $\mathrm{MAD}^{(t)}$ is a median computed from only the $|\mathcal{S}_t|$ active clients sampled that round (six of ten in our experiments), not from the full client population, so it is itself a small-sample statistic whose relative estimation error does not vanish as the true population becomes more homogeneous – if anything, the opposite: as the true dispersion of $\{d_j^{(t)}\}$ shrinks toward the noise floor of ordinary training-run variability (gradient noise, batch composition, early-stopping timing), the six-client sample median of distances becomes dominated by this floor rather than by any meaningful signal about population spread, and a client whose fixed absolute deviation $d_k^{(t)}$ happens not to shrink at the same rate as the sample's median produces an inflated $z_k^{(t)} = d_k^{(t)}/\mathrm{MAD}^{(t)}$ through no fault of its own. We do not present this as a theorem, since the precise rate depends on the (unknown, empirically varying) relationship between true population homogeneity and per-round training noise, but Section 3.2 shows it is severe enough in practice to produce a 62.5% benign false-exclusion rate under IID CICIDS2017 data before correction (Section 5.5), which is the empirical grounding for the MAD-flooring correction described there.

3.2 Problem setting and backbone model

We consider a federated learning system with K clients, of which an unknown subset $\mathcal{M} \subset \{1, \ldots, K\}$ may be malicious. In each communication round t, the aggregator selects a subset $\mathcal{S}_t$ of clients (client_fraction $= 0.6$ of $K = 10$ in our experiments, i.e. six clients per round), broadcasts the current global parameter vector $\theta^{(t)}$, and each selected client $k \in \mathcal{S}_t$ trains locally for up to E epochs with early stopping on a validation-loss patience of P rounds, producing an update $\theta_k^{(t)}$. The aggregator's task is to combine $\{\theta_k^{(t)} : k \in \mathcal{S}_t\}$ into $\theta^{(t+1)}$ such that malicious contributions are suppressed without discarding legitimate heterogeneity.

Our backbone classifier is SE-1DSqueezeNet, reimplemented layer-for-layer from FedSE-1DSqueezeNet [33]: an initial 1D convolution, a depthwise-dilated-squeeze-excite (DDS) module combining a depthwise separable dilated convolution branch with a pointwise convolution branch under a squeeze-and-excitation gate, global average pooling, and a final fully connected classification head. Flow features are treated as a length-F, single-channel sequence. We reuse this backbone without modification, since our contribution is entirely at the aggregation layer; full architectural detail is given in [33] and in our released implementation.

3.3 Baseline aggregation rules

We compare ARMOR-FL against three baselines. **FedAvg** [1] computes a sample-size-weighted average of all selected clients' updates with no robustness mechanism, serving as the undefended floor. **Krum** [2] scores each client by the sum of squared distances to its $k-f-2$ nearest neighbors (where f is the assumed number of Byzantine clients) and selects the single lowest-scoring update as the entire round's contribution. **FoolsGold** [3] tracks the cosine similarity between clients' update histories over time and down-weights clients whose contributions are suspiciously similar to another client's, targeting coordinated Sybil-style attacks specifically rather than independent ones.

3.4 ARMOR-FL aggregation

ARMOR-FL's per-round pipeline (Algorithm 1) proceeds in five stages, three of which are the corrected forms of mechanisms whose naive versions we found, during development, to actively harm robustness under realistic non-IID conditions – we describe each naive version and the specific failure that motivated its replacement, since we believe the failure modes are as reusable a contribution as the fixes.

Robust reference center. Given the active (non-excluded) clients' uploaded vectors, ARMOR-FL computes a coordinate-wise trimmed mean $m^{(t)}$ (trim fraction 0.2) as the round's reference point, and each active client's Euclidean distance to it, $d_k^{(t)} = \|\theta_k^{(t)} - m^{(t)}\|$.

Scale-floored population deviation. The naive population statistic is $z_k^{(t)} = d_k^{(t)}/\mathrm{MAD}^{(t)}$, where $\mathrm{MAD}^{(t)}$ is the median of $\{d_j^{(t)}\}_{j \in \mathcal{S}_t}$ across the round's active clients – a robust scale estimate by construction, since the median of a set of distances divided by itself has median exactly one. The failure we found is that $\mathrm{MAD}^{(t)}$ is itself estimated from only the current round's small, randomly-selected client subset (six of

ten in our experiments), and when the honest population happens to look unusually similar – either because the data is genuinely IID, or simply because that round's random subsample happened to agree closely – $\mathrm{MAD}^{(t)}$ collapses toward zero. Dividing ordinary training noise by a near-zero denominator manufactures large $z_k^{(t)}$ values from clients doing nothing unusual at all. We confirmed this directly on CICIDS2017 under a purely IID partition with 20% malicious clients: the uncorrected statistic excluded 62.5% of honest clients, the highest false-exclusion rate we observed anywhere in the grid, precisely in the setting with the least genuine heterogeneity to explain it. ARMOR-FL floors the scale estimate at a fraction (`mad_floor_fraction`= 0.5) of its own recent round-to-round history, using only the raw, unfloored historical values so the floor cannot ratchet upward on its own account: $\mathrm{MAD}_{\mathrm{eff}}^{(t)} = \max\big(\mathrm{MAD}^{(t)},\ 0.5 \cdot \mathrm{median}(\mathrm{MAD}^{(t-w)}, \ldots, \mathrm{MAD}^{(t-1)})\big)$. Because this can only raise the effective scale relative to the raw value, a genuine attack's raw MAD – which by definition reflects real, large deviation – passes through unaffected; only spurious collapse is corrected.

Each client's population e-process is then updated with $z_k^{(t)}$ via the fixed-mean betting construction of [4]: $e_t = 1 + \lambda(z_k^{(t)} - 1)$ with betting fraction $\lambda = 0.5$ and null mean fixed at one (never recalibrated online, since a client that is already compromised during any calibration window would otherwise learn to appear normal to its own test), and $\log E_{\mathrm{pop},k}^{(t)} = \sum_{s \le t} \log e_s$ accumulates evidence across rounds under the martingale property guaranteed by Ville's inequality.

Self-referential drift and shift tests. A parallel e-process compares each client's current local validation loss to the median and median-absolute-deviation of its own loss history, testing whether the client's own behavior has recently become unstable – this is the mechanism intended, in the original design, to decouple "drifting" (an honest client adapting to an evolving local distribution) from "attacking." We found this insufficient on its own: a client with persistently elevated $z_k^{(t)}$ due to ordinary non-IID heterogeneity typically has a perfectly *stable* own-loss history, since its local data distribution is not changing, only different from the rest of the population – so the drift test never flags it, and the population e-process alone, having no gate on this signal, excludes it anyway. In a controlled synthetic test with ten honest clients (five configured with roughly double the parameter-update variance of the other five, simulating ordinary non-IID scatter) and zero attackers, all five higher-variance clients were excluded within twenty rounds under this design, despite none of them attacking.

ARMOR-FL adds a second self-referential test on the client's own *population-deviation history* rather than its loss: $\mathrm{shift}_k^{(t)} = z_k^{(t)} / \mathrm{median}(z_k^{(t-b)}, \ldots, z_k^{(t-1)})$ against a baseline window of $b = 2$ rounds, testing whether the client has moved relative to *its own* earlier position rather than the population's. A client that has been off-center since the beginning of training produces a shift statistic near one indefinitely and accumulates no shift evidence, however elevated its absolute $z_k^{(t)}$; a client whose deviation appears mid-run – consistent with an attack beginning after an initial clean baseline period, which is how every attack in our experimental grid is actually configured (Section 4) – produces a detectable shift relative to its own pre-attack baseline. Hard exclusion now requires *both* population evidence exceeding the exclusion threshold *and* either shift evidence or a gross-outlier deviation ($z_k^{(t)} \ge 4$, a magnitude no plausible non-IID scatter in our data approached): $\mathrm{exclude}_k^{(t)} = \mathbb{1}\big[E_{\mathrm{pop},k}^{(t)} \ge 1/\alpha_{\mathrm{pop}}\big] \wedge \big(\mathbb{1}[\text{shift evidence}] \vee \mathbb{1}[z_k^{(t)} \ge 4]\big)$. Re-running the same synthetic test with this gate in place produced zero false exclusions across twenty-five rounds with the same five heterogeneous clients.

We are explicit that this gate is not a complete solution and trades one failure mode for a narrower, better-understood one: a client that is Byzantine from round one and remains *constant* thereafter, below the gross-outlier threshold, produces no shift signal either, since nothing about its behavior changes relative to its own (already-compromised) baseline. This is not a bug we failed to fix but a genuine information-theoretic limit – a constant, moderate-magnitude attacker and a heterogeneous-but-honest client are, by definition, indistinguishable from behavior alone in this setting, and resolving the ambiguity would require external information the aggregator does not have. We return to this in Section 8.

Threshold-calibrated trust weighting. The naive smooth downweighting rule, $w_k^{(t)} = n_k \cdot \mathrm{sigmoid}(-\beta \log E_{\mathrm{pop},k}^{(t)})$ with sharpness $\beta = 4$, applies the sigmoid directly to the unbounded log-evidence value. We found this disproportionately punishes clients with only mild deviation: at $\log E_{\mathrm{pop},k}^{(t)} = 1.0$, more than two orders of magnitude below the exclusion threshold of roughly $-\log(0.05) \approx 3.0$, the naive rule already reduces a client's weight to about 2% of its sample-size share. Traced on real CICIDS2017 data under a label-flipping attack, this meant that malicious clients – whose updates, under label-flipping specifically, often happened to look statistically *central* rather than extreme, a known blind spot of any distance-based defense – retained close to full weight throughout training, while honest clients with only ordinary population deviation were crushed to a few percent of their due influence. The net effect handed the undetected attackers *more* relative influence over the aggregate than an unweighted average would have, and ARMOR-FL performed worse than plain FedAvg on this exact scenario as a direct result. ARMOR-FL rescales the sigmoid's input by the same threshold that gates hard exclusion, $\tau = -\log(\alpha_{\mathrm{pop}})$: $w_k^{(t)} = n_k \cdot \mathrm{sigmoid}\big(-\beta(\log E_{\mathrm{pop},k}^{(t)}/\tau - 1)\big)$, giving a trust weight of approximately 0.98 at the population median, 0.5 exactly at the exclusion boundary, and a meaningful drop only for evidence well past it.

Remark 2 (Boundary behavior of the rescaled trust weight) By construction, $w_k^{(t)}/n_k = \mathrm{sigmoid}(0) = 0.5$ exactly when $\log E_{\mathrm{pop},k}^{(t)} = \tau$, i.e. exactly at the population e-process's hard-exclusion boundary – a client is downweighted to precisely half its sample-size share at the same instant it would otherwise be hard-excluded (subject to the shift gate of the preceding paragraph), rather than at some unrelated, separately-tuned threshold. At $\log E_{\mathrm{pop},k}^{(t)} = 0$ (the population median, $E = 1$), $w_k^{(t)}/n_k = \mathrm{sigmoid}(\beta) = \mathrm{sigmoid}(4) \approx 0.982$, so an entirely typical client retains effectively full weight regardless of the specific value of β chosen, provided β is large enough to make $\mathrm{sigmoid}(\beta)$ close to one – the choice of $\beta = 4$ is therefore a sharpness parameter for how quickly weight falls between these two anchored points, not a free parameter that shifts where the anchors themselves sit. This is the precise mechanism by which the correction in this paragraph avoids the naive rule's failure: the two reference points (full trust at the median, half trust at the exclusion boundary) are fixed by the same α_{pop} that governs hard exclusion, rather than by an independently chosen sharpness constant applied to an unbounded raw statistic.

Probation and reinstatement. An excluded client re-enters the federation after a fixed probation window (three rounds in our experiments) at reduced initial trust, with both its population and shift e-processes reset. We note, and discuss further in Section 8, that this reset gives a reinstated attacker a fresh accumulation window before it can be re-excluded, a grace period whose length is bounded by how quickly the population e-process can re-accumulate sufficient evidence.

Algorithm 1 ARMOR-FL aggregation, one round

Input: active client updates $\{\theta_k^{(t)}, n_k, \ell_k^{(t)}\}_{k \in \mathcal{S}_t}$, prior state
$m^{(t)} \leftarrow$ coordinate-wise trimmed mean of $\{\theta_k^{(t)}\}$
$d_k^{(t)} \leftarrow \|\theta_k^{(t)} - m^{(t)}\|$ for each k
$\mathrm{MAD}_{\mathrm{eff}}^{(t)} \leftarrow \max\big(\mathrm{median}(d^{(t)}),\ 0.5 \cdot \mathrm{median}(\text{raw MAD history})\big)$
for each active client k **do**
    $z_k^{(t)} \leftarrow d_k^{(t)}/\mathrm{MAD}_{\mathrm{eff}}^{(t)}$
    update population e-process with $z_k^{(t)}$
    update drift e-process with own-loss deviation
    update shift e-process with $z_k^{(t)}/\mathrm{median}(\text{own } z \text{ history})$
    $w_k^{(t)} \leftarrow n_k \cdot \mathrm{sigmoid}\big(-\beta(\log E_{\mathrm{pop},k}^{(t)}/\tau - 1)\big)$
    if population evidence $\geq$ threshold **and** (shift evidence **or** $z_k^{(t)} \geq 4$) **then**
        exclude k; begin probation; $w_k^{(t)} \leftarrow 0$
    end if
end for
$\theta^{(t+1)} \leftarrow$ weighted average of $\{\theta_k^{(t)}\}$ with weights $\{w_k^{(t)}\}$
Output: $\theta^{(t+1)}$

3.5 Computational complexity

Let K be the number of active clients in a round and d the dimension of the parameter vector θ. FedAvg computes a single weighted mean, $O(Kd)$. The coordinate-wise trimmed mean underlying ARMOR-FL's reference center is likewise $O(Kd \log K)$ once a per-coordinate sort is included, dominated in practice by the $O(Kd)$ term for the client counts used here. Computing each client's distance to the reference center and updating its e-processes is $O(d)$ per client, $O(Kd)$ total, plus $O(K)$ bookkeeping for the MAD history and shift baselines – ARMOR-FL's aggregation-layer overhead over FedAvg is therefore $O(Kd)$, the same asymptotic order as FedAvg itself. Krum requires the full pairwise distance matrix between all K active clients, $O(K^2 d)$, to compute each client's $k-f-2$-nearest-neighbor score; this quadratic-in-K term is the asymptotically most expensive of the four rules, though it remains modest at the client counts we test ($K = 10$, six active per round). FoolsGold additionally maintains a full pairwise cosine-similarity history across rounds, $O(K^2)$ per round on top of the $O(Kd)$ update step.

This ordering is reflected in the wall-clock measurements taken during the robustness grid (Table 9, Section 5.6): Krum is, in practice, the slowest of the four aggregators we test, not ARMOR-FL, despite the latter's additional per-client sequential-testing machinery – an aggregation-layer cost that turns out to be small relative to local training time (which dominates total wall-clock time for all four rules) and relative to Krum's pairwise-distance computation specifically once neighbor counts and parameter dimension are large enough for the $O(K^2 d)$ term to matter. We view this as a relevant practical counterpoint to Section 5.1's headline finding that Krum wins on accuracy: Krum's accuracy advantage does not come for free, and a deployment sensitive to per-round latency, not only to final accuracy, may still prefer ARMOR-FL's aggregation cost profile even where its accuracy trails Krum's.

4 Experimental Setup

4.1 Datasets

We evaluate on three widely used network intrusion detection datasets. **CICIDS2017** [29] contains five days of labeled traffic covering benign activity and a range of attack types (DoS, DDoS, brute force, infiltration, web attacks, botnet) captured on a realistic network testbed. **CICIDS2018** extends the same collection methodology across a substantially larger ten-day capture. **CICIoT2023** [30] is a more recent, IoT-focused dataset covering 33 attack types across seven categories, collected from a 105-device smart-home testbed; we are not aware of any prior Byzantine-robust federated defense reporting a poisoning-attack evaluation on it. All datasets are partitioned across ten simulated clients using either an IID split or a Dirichlet-distributed non-IID split with concentration parameter $\alpha \in \{0.5, \text{IID}\}$; smaller α produces more skewed, less similar per-client class distributions. Reflecting the practical compute budget available for the full experimental grid, we sample 10% of CICIDS2017, 2% of CICIDS2018, and 1% of CICIoT2023 for the robustness grid, chosen so that CICIDS2017's benchmark run-time exactly calibrates the sampling fraction for the other two; the resulting training-set sizes remain in the hundreds of thousands of rows.

4.2 Attacks

We evaluate three attack types spanning increasing subtlety. **Label-flipping** is a data-space attack in which a malicious client trains honestly on relabeled data, producing a parameter update that is statistically difficult to distinguish from an honest one – a known blind spot shared by every distance-based defense we test, not unique to ARMOR-FL. **Gaussian-noise** is a parameter-space attack that perturbs an otherwise-honest update with additive noise, typically producing a visibly larger and more detectable deviation. **ALIE** (a little is enough) [48] is an adaptive, omniscient attack that crafts a perturbation calibrated to stay within the expected statistical spread of the honest population, designed specifically to evade distance-based outlier detection. Attacks begin at round three of each run (`attack_start_round`= 3), after a two-round

clean baseline – a design choice that matters for ARMOR-FL's shift test specifically, since it guarantees every client has an honest baseline period before any attack can begin, which is what makes the shift signal meaningful rather than vacuous. We test malicious client fractions of 0.2 and 0.4 (two and four of ten clients, respectively).

4.3 Attack implementation details

We give the exact per-attack construction here, since Section 5 and Section 6 draw specific conclusions (particularly the CICIoT2023 Gaussian-noise collapse, Section 5.8, and label-flip's central-looking updates, Section 3) that depend on these details. **Label-flipping** applies a deterministic class shift to a malicious client's local labels before training, $y \mapsto (y + 1) \bmod C$ for C classes, so the client trains an otherwise-ordinary supervised update on systematically mislabeled data; no perturbation is applied to the resulting parameter update itself, which is precisely why it remains statistically close to an honest update (Section 3). **Gaussian-noise** replaces a malicious client's entire update with pure Gaussian noise, re-scaled to match the norm of what an honest update from the same client would have had: $\Delta_{\text{malicious}} = \sigma \cdot \|\Delta_{\text{honest}}\| \cdot \eta / \|\eta\|$ for $\eta \sim \mathcal{N}(0, I)$ and noise scale $\sigma = 1.0$; norm-matching is a deliberate design choice so the attack cannot be caught by a trivial magnitude-only check, and it is also, we now believe (Section 5.8), the specific reason the attack proved catastrophic on CICIoT2023: a norm-matched *random-direction* replacement discards all of the honest gradient signal while preserving its scale, which is more destructive on a harder classification task where a larger share of the update's norm carries genuine task-relevant signal. **ALIE** [48] computes the coordinate-wise mean and standard deviation across the current round's honest updates and returns $\Delta_{\text{malicious}} = \bar{\Delta}_{\text{honest}} - z_{\max} \cdot \sigma_{\text{honest}}$ with $z_{\max} = 1.5$, an omniscient-attacker construction (it requires visibility into the other honest clients' updates the same round, standard in the ALIE literature) designed to bias the aggregate while staying within the honest population's own observed spread.

4.4 Hyperparameters and deviations from the original protocol

Table 2 lists the federated-learning hyperparameters used across the robustness grid. We follow FedSE-1DSqueezeNet's own protocol [33] where feasible, with one disclosed deviation: batch size. The original protocol specifies a batch size of 32; we found, in a controlled sweep on identical data and hyperparameters otherwise, that per-round wall-clock time dropped from 211.5 seconds at batch size 32 to 25.9 seconds at batch size 256, an approximately eightfold reduction, with no measurable change in final accuracy or F1 across the batch sizes tested (32, 128, 256, 512). We attribute this to the backbone model being small enough, and the GPU used for these experiments large enough, that batch size 32 leaves the hardware overwhelmingly latency-bound rather than compute-bound – GPU utilization at batch size 32 measured between 5% and 18% across a representative benchmark run. We use batch size 256 throughout the robustness grid on this basis, a practical compute-budget decision rather than a change to the federated-learning protocol itself, and disclose it here rather than presenting it silently as the original protocol's own setting. Separate comparability runs reproducing the original paper's own R=5, FedAvg-only, IID-plus-three-non-IID-level sanity check retain batch size 32 throughout, since their purpose is fidelity to the original protocol rather than robustness-grid throughput.

4.5 Evaluation metrics

We report two accuracy statistics per run: **final-round accuracy**, the standard metric and the one directly comparable across all four aggregators, and **mean accuracy over the final eight rounds**, a secondary, more stable statistic we introduce after observing that individual runs oscillate considerably round to round – a property of the random six-of-ten client subsampling used every round, not unique to ARMOR-FL (Section 5). We additionally report the **stability gap**, $|\,\text{final-round accuracy} - \text{mean last-8-round accuracy}\,|$, as a compact summary of how much a single reported final-round number can mislead relative to the run's recent trend.

Table 2 Federated learning hyperparameters, robustness grid

Parameter	Value
Clients (K)	10
Client fraction per round	0.6
Communication rounds	25
Local epochs (max, with early stopping)	10
Local early-stopping patience	3
Learning rate	0.01
Batch size	256 (see Section 4.4)
Dropout	0.5
Attack start round	3
ARMOR-FL burn-in rounds	3
ARMOR-FL α_{pop}, α_{drift}, α_{shift}	0.05 each
ARMOR-FL trim fraction	0.2
ARMOR-FL MAD floor fraction	0.5
ARMOR-FL gross-outlier threshold (z)	4.0
ARMOR-FL probation rounds	3

For ARMOR-FL specifically, we additionally report three detection statistics computed against the ground-truth malicious client labels available in simulation (not available to the aggregator itself, which sees only behavior): **detection precision**, the fraction of clients ARMOR-FL ever excludes over a run that are truly malicious; **detection recall**, the fraction of truly malicious clients ARMOR-FL excludes at least once; and **benign false-exclusion rate**, the fraction of truly honest clients ARMOR-FL excludes at least once. All three are averaged first within a run and then across the twelve attack-fraction-α combinations per dataset, matching the aggregation used for accuracy in Table 3.

4.6 Ablation protocol

To isolate the effect of the three corrections described in Section 3 (the self-shift e-process, the scale-floored population statistic, and the threshold-calibrated trust weight) from the aggregation mechanism's design in general, we retain the full robustness-grid results produced by an earlier, uncorrected implementation of ARMOR-FL, run under identical hyperparameters, datasets, attacks, and malicious-fraction/non-IID-α combinations to the corrected grid reported in Section 5. We refer to this earlier implementation as **ARMOR-FL (uncorrected)** throughout Section 5.5 and report it exclusively as an ablation baseline against the corrected method, ARMOR-FL (ours) elsewhere in this paper – it is not proposed as a competing method and its aggregate accuracy numbers, where they appear, should be read only in comparison to the corrected version's, not against Krum, FoolsGold, or FedAvg.

5 Results

5.1 Aggregate accuracy across datasets

Table 3 reports mean final-round accuracy and mean last-eight-round accuracy, averaged across all twelve attack-type × malicious-fraction × non-IID-α combinations, for each aggregator on each dataset.

Krum achieves the highest aggregate mean accuracy on all three datasets, whether measured at the final round or averaged over the last eight rounds. Against the two remaining baselines the picture is mixed rather than uniformly favorable, and we report it precisely rather than rounding it up: ARMOR-FL trails FedAvg on CICIDS2017 (0.695/0.714 vs. 0.727/0.727) and CICIDS2018 (0.793/0.731 vs. 0.825/0.787), and is essentially tied with it on CICIoT2023 (0.423 vs. 0.413 at the final round, but 0.422 vs. 0.427 on the last-eight-round mean – a wash, not a genuine advantage either way). Against FoolsGold, ARMOR-FL trails on CICIDS2017 (0.695/0.714

Table 3 Mean accuracy (final-round / last-8-round mean) across all twelve attack-fraction-α combinations per dataset

Aggregator	CICIDS2017	CICIDS2018	CICIoT2023
FedAvg (undefended)	0.727 / 0.727	0.825 / 0.787	0.413 / 0.427
Krum	**0.810** / **0.781**	**0.810** / **0.809**	**0.618** / **0.623**
FoolsGold	0.715 / 0.720	0.718 / 0.740	0.378 / 0.381
ARMOR-FL (ours)	0.695 / 0.714	0.793 / 0.731	0.423 / 0.422

vs. 0.715/0.720), is mixed on CICIDS2018 (ahead at the final round, 0.793 vs. 0.718, but behind on the last-eight-round mean, 0.731 vs. 0.740), and is clearly ahead on CICIoT2023 (0.423/0.422 vs. 0.378/0.381). ARMOR-FL does not surpass Krum's aggregate mean on any dataset by either statistic. We report this plainly: this is not a result in which our proposed method dominates across the board, and Section 6 discusses why Krum's winner-take-all selection mechanism appears structurally better suited to this particular attack grid than ARMOR-FL's weighted-averaging approach, along with the specific conditions under which the reverse is true.

All three datasets show meaningful oscillation between final-round and last-eight-round accuracy for every aggregator except Krum, confirming that this instability is a general property of the six-often random client subsampling used every round rather than something specific to ARMOR-FL's own mechanism; Krum's per-round winner-take-all selection is evidently more robust to this sampling noise than any of the three averaging-based rules.

5.2 Per-attack-type breakdown

Table 3 aggregates across three attack types that, as Section 4 notes, differ substantially in how detectable they are from parameter-space distance alone. Table 4 disaggregates final-round accuracy by attack type, revealing patterns the aggregate obscures.

Three patterns stand out. First, ARMOR-FL beats Krum on label-flip specifically on two of the three datasets (CICIDS2018 and CICIoT2023), consistent with the case study in Section 5.7: label-flipping is the attack type where our self-shift mechanism has genuine signal to exploit, since a label-flipping client's parameter update looks statistically central under a pure population comparison but still produces a detectable shift relative to its own pre-attack baseline. Second, under Gaussian-noise on CICIDS2017 and CICIDS2018, all four aggregators – including undefended FedAvg – achieve nearly identical accuracy, indicating that at the malicious fractions we test (0.2 and 0.4), additive parameter-space noise of the magnitude we inject is not, on these two datasets, disruptive enough to differentiate a defended aggregator from an undefended one; robustness mechanisms have no useful role to play when the undefended baseline itself is not meaningfully harmed. Third, and starkly, Gaussian-noise on CICIoT2023 produces a catastrophic collapse to 0.023 accuracy for every aggregator except Krum, which retains 0.737 – we discuss this specific result as its own case study in Section 5.8, since it is the single largest accuracy gap between any two aggregators anywhere in our grid.

5.3 Per-malicious-fraction and non-IID-α breakdown

Table 5 further disaggregates final-round accuracy by malicious client fraction (0.2 versus 0.4) and by non-IID Dirichlet concentration ($\alpha = 0.5$ versus IID), each averaged across the remaining grid dimensions.

ARMOR-FL wins its single row on CICIDS2017 (mal. frac. 0.2, where the malicious presence is lightest) but loses ground sharply as malicious fraction rises to 0.4 on the same dataset – a large drop from 0.852 to 0.538, larger than any other aggregator's drop across the same transition. This is consistent with the mechanical explanation offered in Section 6: ARMOR-FL's trust-weighted averaging blends in every insufficiently-suppressed malicious contribution, so a higher malicious fraction directly increases the number of potentially-blended-in attackers each round, whereas Krum's winner-take-all rule is comparatively insensitive to how many attackers are present as long as at least

Table 4 Final-round accuracy by dataset and attack type, mean across malicious-fraction and non-IID-α combinations

Dataset	Attack	FedAvg	Krum	FoolsGold	ARMOR-FL
CICIDS2017	ALIE	0.791	**0.816**	0.791	0.690
	Gaussian-noise	**0.803**	0.757	**0.803**	**0.803**
	Label-flip	0.586	**0.857**	0.549	0.591
CICIDS2018	ALIE	0.824	**0.872**	0.831	0.820
	Gaussian-noise	0.831	**0.893**	0.831	0.831
	Label-flip	**0.820**	0.666	0.492	0.729
CICIoT2023	ALIE	**0.767**	0.538	0.554	0.598
	Gaussian-noise	0.023	**0.737**	0.023	0.023
	Label-flip	0.449	**0.579**	0.556	**0.647**

Table 5 Final-round accuracy by malicious fraction and by non-IID α, mean across the remaining grid dimensions

Dataset	Condition	FedAvg	Krum	FoolsGold	ARMOR-FL
CICIDS2017	mal. frac. 0.2	0.823	0.795	0.791	**0.852**
	mal. frac. 0.4	0.630	**0.825**	0.638	0.538
CICIDS2018	mal. frac. 0.2	0.833	**0.888**	0.735	0.829
	mal. frac. 0.4	**0.816**	0.733	0.701	0.758
CICIoT2023	mal. frac. 0.2	0.513	**0.728**	0.527	0.552
	mal. frac. 0.4	0.314	**0.508**	0.229	0.294
CICIDS2017	$\alpha = 0.5$ (non-IID)	0.750	**0.797**	0.669	0.706
	IID	0.703	**0.823**	0.760	0.683
CICIDS2018	$\alpha = 0.5$ (non-IID)	0.819	0.703	0.684	**0.789**
	IID	0.831	**0.917**	0.752	0.798
CICIoT2023	$\alpha = 0.5$ (non-IID)	0.425	**0.671**	0.406	0.476
	IID	0.402	**0.566**	0.350	0.370

one clearly honest client remains among the six sampled. Krum wins seven of the twelve rows in Table 5 outright, and ARMOR-FL's one clear win (CICIDS2018, $\alpha = 0.5$) is again a non-IID condition, reinforcing that ARMOR-FL's practical niche is non-IID rather than IID data, where our own diagnostic work (Section 3) shows population-only defenses are least reliable to begin with.

5.4 Detection statistics

Table 6 reports ARMOR-FL's own detection precision, detection recall, and benign false-exclusion rate, averaged across all twelve combinations per dataset. Detection precision remains low throughout (0.167 to 0.312), meaning a meaningful fraction of clients ARMOR-FL excludes over a full run are not actually malicious, even after all three corrections described in Section 3; benign false-exclusion rates of 0.125 to 0.163 confirm the same pattern from a different angle. Detection recall is lower still (0.062 to 0.229): ARMOR-FL frequently fails to exclude a truly malicious client at all over the course of a run, not only misfires against honest ones. This is a genuine remaining limitation, not one hidden by the aggregate accuracy numbers in Table 3, and we discuss its likely causes – the reinstatement grace period and label-flipping's fundamental resistance to distance-based detection – in Section 8.

Table 7 breaks detection statistics down further by attack type. Gaussian-noise, the most visibly extreme parameter-space perturbation of the three, is the attack type ARMOR-FL detects

Table 6 ARMOR-FL detection statistics, mean across all twelve combinations per dataset

Dataset	Precision	Recall	Benign FER
CICIDS2017	0.312	0.229	0.125
CICIDS2018	0.167	0.062	0.163
CICIoT2023	0.194	0.125	0.142

most precisely when it detects it at all (precision up to 1.0 on CICIDS2018), but its recall on the same attack type is frequently zero – the aggregation-layer collapse described in Section 5.2 and 5.8 happens too fast, on CICIoT2023 specifically, for the sequential test to accumulate enough rounds of evidence before the damage is done. ALIE, designed explicitly to evade distance-based detection [48], produces the highest benign false-exclusion rates of the three attack types on two of three datasets (0.198–0.271), suggesting that an attack calibrated to blend into the honest population's statistical spread also makes it harder for the population e-process to separate genuinely elevated honest heterogeneity from the attack's own camouflaged elevation – an adaptive attacker narrowing the very margin ARMOR-FL's self-shift test depends on.

5.5 Ablation: corrected versus uncorrected ARMOR-FL

Section 4.6 describes an archived grid run under an earlier, uncorrected implementation of ARMOR-FL – before the self-shift e-process, MAD-flooring, and threshold-calibrated trust weight of Section 3 were applied – run under identical conditions to the corrected grid. Table 8 compares the two directly. The corrected version reduces the benign false-exclusion rate on every dataset (most sharply on CICIDS2017, from 0.309 to 0.125, a reduction of more than half), confirming quantitatively, on the full real-data grid and not only the isolated synthetic reproductions of Section 3, that the three corrections do what they were designed to do. The effect on aggregate final-round accuracy is more mixed – an improvement on CICIoT2023 (0.395 to 0.423) and CICIDS2017 (0.689 to 0.695), a small regression on CICIDS2018 (0.805 to 0.793) – which we read as consistent with the honest framing of this paper throughout: reducing false exclusions is not the same lever as maximizing final accuracy, and the corrections were motivated by, and validated against, the false-exclusion pathology specifically rather than an aggregate accuracy target.

The uncorrected version's detection precision is, counter-intuitively, higher than the corrected version's on two of three datasets. We attribute this to a direct trade-off rather than a regression: the uncorrected version excludes more clients overall (both honest and malicious, hence its higher false-exclusion rate), and a subset of those additional exclusions happen to be correct, inflating precision on a smaller, more aggressively-filtered set of exclusion events. The corrected version excludes far fewer clients in total, and among a smaller exclusion set, an equal or larger absolute number of correct detections would be needed to preserve precision – Section 8 returns to this trade-off between exclusion aggressiveness and exclusion accuracy as an open design tension we did not fully resolve.

5.6 Computational overhead

Table 9 reports mean wall-clock time per full run (25 communication rounds), averaged across all twelve attack-fraction-α combinations per dataset, for each aggregator. Consistent with the complexity analysis in Section 3.5, Krum is the slowest of the four aggregators in aggregate, not ARMOR-FL – ARMOR-FL's additional sequential-testing bookkeeping adds negligible measured overhead over undefended FedAvg (within noise on two of three datasets), while Krum's pairwise-distance computation adds a consistent, and on CICIDS2018 substantial, overhead.

This is a relevant counterpoint to Section 5.1's headline result: Krum's accuracy advantage over ARMOR-FL comes with a non-trivial, consistent computational cost – 38% higher aggregate wall-clock time in our measurements – that a latency-sensitive deployment (real-time network intrusion detection is, definitionally, latency-sensitive) may not be willing to pay for the accuracy gained, particularly given that Section 5.3 shows Krum's

Table 7 ARMOR-FL detection statistics by attack type, mean across malicious-fraction and non-IID-α combinations

Dataset	Attack	Precision	Recall	Benign FER
CICIDS2017	ALIE	0.222	0.250	0.198
	Gaussian-noise	0.500	0.250	0.031
	Label-flip	0.278	0.188	0.146
CICIDS2018	ALIE	0.000	0.000	0.271
	Gaussian-noise	1.000	0.125	0.000
	Label-flip	0.083	0.062	0.219
CICIoT2023	ALIE	0.167	0.250	0.229
	Gaussian-noise	0.000	0.000	0.062
	Label-flip	0.333	0.125	0.135

Table 8 Ablation: uncorrected versus corrected ARMOR-FL, mean across all twelve combinations per dataset

Dataset	Version	Final accuracy	Precision	Benign FER
CICIDS2017	Uncorrected	0.689	0.288	0.309
	Corrected (ours)	**0.695**	**0.312**	**0.125**
CICIDS2018	Uncorrected	**0.805**	**0.352**	0.188
	Corrected (ours)	0.793	0.167	**0.163**
CICIoT2023	Uncorrected	0.395	0.358	0.208
	Corrected (ours)	**0.423**	0.194	**0.142**

accuracy advantage narrows or reverses under specific non-IID, lighter-poisoning conditions where ARMOR-FL's mechanism is strongest.

Our measured wall-clock numbers are for $K = 10$ clients (six active per round), the scale at which local training dominates total per-round time (Section 3.5) and the aggregation-layer cost difference between rules is a comparatively small share of the total. We have not run experiments at larger K, so the following is a theoretical projection of the *aggregation step's own asymptotic cost* from Section 3.5's complexity analysis, not an empirically measured extrapolation, and it deliberately ignores local-training cost, which would dominate total wall-clock time at any K and does not differ between aggregation rules. Under the $O(K^2 d)$ versus $O(Kd)$ asymptotic forms of Table 1, Krum's aggregation-layer cost relative to ARMOR-FL's grows linearly in the number of active clients per round: holding parameter dimension d fixed, a federation with ten times as many active clients per round (60 rather than six) would see Krum's pairwise-distance computation grow roughly tenfold relative to ARMOR-FL's linear update, and a hundredfold increase in active clients projects to a hundredfold relative growth in this specific cost component. At the scale of a real multi-tenant SOC federation (hundreds to low thousands of participating clients), this projects Krum's aggregation-layer cost advantage for ARMOR-FL to become substantially more pronounced than the modest 38% total-wall-clock gap measured at $K = 10$ – though whether this aggregation-layer cost becomes large enough, relative to local training time at that same scale, to matter for an operator's actual per-round latency budget is a question our experiments do not answer and that we flag as a concrete, currently untested claim for future work at realistic production scale.

5.7 Case study: label-flipping under non-IID data at low malicious fraction

The setting that most clearly illustrates when ARMOR-FL's mechanism earns its keep is label-flipping under non-IID data ($\alpha = 0.5$) with 20% of clients malicious, on CICIDS2017. Here ARMOR-FL achieves 0.891 final-round accuracy (mean

Table 9 Mean wall-clock time per 25-round run (seconds), averaged across all twelve combinations per dataset

Dataset	FedAvg	Krum	FoolsGold	ARMOR-FL
CICIDS2017	243.0	**278.3**	240.7	245.7
CICIDS2018	285.5	**577.5**	358.8	306.1
CICIoT2023	429.1	**467.9**	395.6	397.4
Overall mean	319.2	**441.2** (+38%)	331.7 (+4%)	316.4 (−1%)

last-8-round accuracy 0.715), ahead of Krum (0.811 final-round, 0.686 last-8-mean), FoolsGold (0.735), and undefended FedAvg (0.799 final-round, only 0.402 last-8-mean – FedAvg's apparently competitive final-round number masks a much less stable trend, one of the clearest illustrations in our grid of why we report both statistics, Section 4.5). Under the same attack and malicious fraction but IID data, ARMOR-FL's advantage disappears entirely: Krum takes over (0.904 final-round, 0.836 last-8-mean, against ARMOR-FL's 0.864 and 0.721 respectively). This reversal is precisely consistent with the mechanism described in Section 3: ARMOR-FL's shift-based exclusion gate is most effective precisely when there is a genuine, attack-coincident change to detect against a stable non-IID baseline, and least effective when the population itself provides little contrast between honest heterogeneity and attack, and its advantage is reported here only at the malicious fraction (0.2) and non-IID setting where Table 5 (Section 5.3) shows it holds in aggregate as well, not as an isolated favorable round.

We flag directly, rather than let the reader discover it independently, that this specific margin is a single-seed result (Section 7, internal validity) and is not large relative to the round-to-round oscillation this paper documents elsewhere: the last-8-mean margin over Krum here (0.715 vs. 0.686, a difference of 0.029) is smaller than the stability gap FedAvg exhibits in this very case study (0.799 final-round vs. 0.402 last-8-mean, a swing of 0.397), and we cannot rule out that a different random seed would narrow, erase, or reverse this specific margin. What we consider robust to this concern is not the exact size of the win in this one cell, but the aggregate pattern across the twelve-cell grid (Table 5) that malicious fraction 0.2 under non-IID data is the one row where ARMOR-FL leads on this dataset, and the mechanistic account of why (Section 3) that this case study is chosen to illustrate concretely rather than to prove on its own. We therefore present this case study as an illustration of the mechanism consistent with the aggregate pattern, not as standalone statistical evidence that the shift mechanism is responsible for this exact margin; Section 7 discusses the multi-seed replication this would need to confirm.

5.8 Case study: catastrophic Gaussian-noise collapse on CICIoT2023

Table 4 (Section 5.2) reports a striking result under Gaussian-noise on CICIoT2023: FedAvg, FoolsGold, and ARMOR-FL all collapse to 0.023 final-round accuracy – indistinguishable from random guessing on this multi-class problem – while Krum retains 0.737, the single largest gap between any two aggregators anywhere in our experimental grid. We examine this case separately because it is the clearest illustration of a qualitatively different failure than the one this paper otherwise addresses: this is not a false-exclusion problem, nor a detection-precision problem, but an outright aggregation collapse.

The per-round trace for this scenario shows all three collapsing aggregators reaching near-zero accuracy within the first two to three post-attack rounds and never recovering, rather than degrading gradually as malicious influence accumulates. We attribute this to the specific interaction between CICIoT2023's already-difficult multi-class classification task (Section 6) and additive Gaussian perturbation at a fixed noise scale calibrated relative to CICIDS2017/2018 parameter norms: on a harder task, the backbone's parameter updates plausibly occupy a narrower useful region of weight space to begin with, so an additive perturbation of a given absolute magnitude is proportionally more destructive there than on an easier, better-separated classification problem. Once a single round's aggregate crosses into a sufficiently

corrupted region of weight space, subsequent rounds of otherwise-honest local training on top of a badly corrupted global model cannot recover in the 25 rounds our grid budgets, for any of the three averaging-based rules. Krum's winner-take-all selection sidesteps this entirely: by discarding every other client's contribution outright, a single successfully-identified honest client's update alone determines the round's aggregate, which cannot itself be corrupted by another client's additive noise regardless of that noise's magnitude.

This case study qualifies the wall-clock cost-benefit argument of Section 5.6: Krum's computational overhead may be a reasonable price specifically as insurance against this kind of catastrophic, non-recoverable collapse, even where its typical-case accuracy advantage over a weighted-averaging rule is comparatively modest. It also identifies a concrete direction for future work on ARMOR-FL specifically: a hard per-round bound on how much any single round's aggregate can move relative to the previous round's – a form of trust-region constraint independent of the sequential-testing mechanism – might prevent this class of one-round catastrophic corruption without requiring Krum's winner-take-all discarding of all other legitimate contributions, though we have not implemented or validated such a mechanism and leave it to future work.

6 Discussion

The central empirical finding of this paper is uncomfortable in a specific way: we built ARMOR-FL to fix a real, previously undiagnosed failure mode in population-only Byzantine defenses, confirmed the fix works exactly as intended on the failure mode it targets, and still find that Krum – a much simpler, purely empirical, winner-take-all selection rule with no statistical guarantee at all – outperforms it in aggregate on every dataset we tested. We think this is worth sitting with rather than explaining away, and devote this section to unpacking why, when the advantage reverses, and what a practitioner should take from both facts together.

6.1 Why Krum wins in aggregate

Part of the explanation is mechanical. Krum discards all but one client's contribution every round, so a single successfully identified honest client is enough to keep the aggregate clean; ARMOR-FL instead blends contributions from potentially many clients via trust-weighted averaging, which is the more statistically efficient choice when all contributors are honest but means that even one undetected or under-suppressed malicious contributor's update is blended into the aggregate every round it participates. Under label-flipping specifically, where a malicious client's update often looks statistically central rather than extreme (Section 3), this blending cost is at its highest, since the malicious contribution retains substantial weight for many rounds at a stretch. Table 5 (Section 5.3) shows this cost growing sharply with malicious fraction: ARMOR-FL's advantage over Krum at malicious fraction 0.2 on CICIDS2017 reverses into a substantial deficit at 0.4, precisely as the mechanical account predicts – more potential blended-in attackers per round directly increases the averaging rule's exposure.

6.2 When ARMOR-FL wins

The case study in Section 5.7 shows the reverse relationship holds under the right conditions: non-IID data plus a lighter malicious fraction is exactly the regime where a stable per-client baseline gives ARMOR-FL's shift test real signal to work with before a larger share of any given round's six selected clients being malicious starts to overwhelm the trust-weighted average (Section 5.3 shows this advantage reversing sharply once malicious fraction rises to 0.4). Table 4 reinforces this at the level of attack type specifically: ARMOR-FL beats Krum on label-flip on two of three datasets, the attack type our method's self-shift mechanism is best positioned to exploit given label-flipping's central-looking parameter updates. We take this as the honest scope of ARMOR-FL's practical advantage: it is not a universal improvement over Krum, but a targeted one, strongest exactly where formal, cumulative evidence-gathering has the most to offer over an isolated per-round heuristic – non-IID populations, label-flipping-style attacks, and lighter poisoning specifically, before a heavier malicious presence overwhelms the averaging-based mechanism (Section 6.1).

6.3 The CICIoT2023 difficulty confound

CICIoT2023's uniformly lower accuracy across all four aggregators (Table 3) is itself worth discussing rather than treating as noise. CICIoT2023 is a substantially harder multi-class problem than either CIC dataset – our own earlier centralized baseline check (an XGBoost classifier with full access to pooled training data and no federated partitioning at all) reached only 0.856 accuracy and 0.660 macro-F1 on CICIoT2023, against 0.999 and 0.932 on CICIDS2017 – so a meaningful share of the accuracy gap between aggregators on CICIoT2023 likely reflects the underlying classification difficulty of the dataset itself rather than aggregator robustness specifically, and Krum's relative advantage there (0.618 versus ARMOR-FL's 0.423) may partly reflect that a winner-take-all rule is less sensitive to a harder base classification problem's own noise than a weighted average is. The catastrophic Gaussian-noise collapse documented in Section 5.8 is consistent with this reading: a harder underlying task appears to leave less margin in weight space for any averaging-based rule to absorb even one round of unsuppressed additive noise.

6.4 Practical deployment guidance

Bringing Sections 5.3, 5.6, and 6.2 together, we offer the following guidance, but we state its limits plainly before stating the guidance itself: it is descriptive of the regime where our experiments show ARMOR-FL's mechanism helps, not a prospective decision procedure a practitioner can run before deployment. In particular, attack type is, by the nature of Byzantine robustness, adversarially chosen and not knowable in advance – a deployment cannot simply check whether it will face label-flipping before choosing an aggregator. Non-IID-ness, by contrast, *is* estimable in advance and without knowledge of any attack, e.g. from historical per-client class-distribution telemetry or a Dirichlet-concentration proxy computed on past (presumed largely honest) training rounds; a deployment with an already-known non-IID client population can act on that half of the guidance directly. For the attack-type half, we can only offer a retrospective framing: an operator whose incident history or threat model makes label-flipping (insider mislabeling, compromised data pipelines) a plausible dominant vector, and who separately values a formal, auditable statistical guarantee on false-exclusion behavior (Section 6.5 discusses precisely what that guarantee does and does not promise), is the setting where ARMOR-FL's mechanism earns a genuine advantage over Krum, at lower aggregation-layer computational cost (Section 5.6). A deployment where malicious fraction may be high, where Gaussian-noise-style parameter-space corruption is a plausible threat, or where an operator's risk tolerance favors bounding worst-case damage from a single catastrophic round over average-case accuracy, is better served by Krum's winner-take-all rule despite its higher wall-clock cost and complete absence of a statistical guarantee – and we note explicitly that guessing wrong carries asymmetric downside: Section 5.8 shows an averaging-based rule including ARMOR-FL can collapse to near-random accuracy in a single unfavorable scenario (CICIoT2023 under Gaussian-noise), whereas the downside of defaulting to Krum where ARMOR-FL was in fact preferable is bounded to the modest accuracy gap documented in Table 5. A risk-averse operator uncertain which regime applies should weight this asymmetry, not only the average-case comparison, when deciding whether the added implementation complexity of ARMOR-FL is worth it. We do not consider these recommendations final; they follow from a specific, disclosed experimental grid (Section 4) and should be revisited as that grid is extended (Section 9).

6.5 Relation to the anytime-valid inference literature

A broader point this paper's results raise for the anytime-valid inference literature specifically: a formal statistical guarantee on Type-I error (Theorem 1) is a guarantee about *false-exclusion control*, not about *final task performance*, and this paper's results show these two objectives can diverge substantially in a real deployment. We first reconcile a gap a careful reader will already have noticed: Theorem 1 is stated at significance level $\alpha_{\text{pop}} = 0.05$, yet Table 6 reports realized benign false-exclusion rates of 0.125–0.163, two-and-a-half to three times higher than the nominal bound. This is not a violation of Theorem 1 – the theorem is unconditionally true given its hypotheses – but

a direct consequence of those hypotheses not holding exactly for the clients the bound is meant to protect. The theorem controls the false-rejection probability under a null hypothesis of *exactly* $\mu_0 = 1$; a genuinely heterogeneous honest client's true $z_k^{(t)}$ mean is typically somewhat above one (that is precisely the non-IID heterogeneity this paper is about), so the null the guarantee formally protects is already mildly misspecified for exactly the clients whose false exclusion we most want to prevent, and the reported benign FER additionally aggregates across many clients, rounds, and grid combinations rather than reporting one client's single-sequence Type-I rate, a different quantity than what Theorem 1 directly bounds. We state this plainly rather than letting the $\alpha_{\text{pop}} = 0.05$ figure imply a real-world guarantee it does not deliver: the corrected design substantially reduces false exclusion relative to the uncorrected baseline (Section 5.5), and Theorem 1 is mathematically correct as stated, but neither claim should be read as promising a 5% real-world false-exclusion rate, and Section 8 already reports the realized rate as an open, unresolved cost rather than a solved problem.

ARMOR-FL succeeds unambiguously at the false-exclusion-*reduction* objective relative to its own uncorrected predecessor – Section 5.5's ablation shows a large, consistent reduction in benign false-exclusion rate relative to an uncorrected population-only test. It does not, on the evidence in Table 3, translate into the strongest final-accuracy outcome in aggregate. We think this is a useful cautionary note for anytime-valid methods entering applied machine-learning venues more broadly, where a formal guarantee is sometimes implicitly treated as synonymous with practical superiority: the guarantee is real and valuable in its own right (auditability, principled false-exclusion control, no need to pre-commit to a stopping time), but it is a different kind of claim from a task-performance claim, and a paper introducing an anytime-valid method should report both, and let a reader who cares primarily about the former accept a method that trails on the latter – exactly the reporting stance we have tried to take throughout this paper.

7 Threats to Validity

We separate four categories of threat to the conclusions drawn above, following standard empirical machine-learning reporting practice, since this paper's results are ultimately an empirical claim about a specific, disclosed experimental grid rather than a universal one.

Internal validity. All results in this paper come from a single random seed per configuration, not repeated trials with confidence intervals; the meaningful round-to-round oscillation documented in Section 5.1 and the stability-gap statistics in Table 3 are evidence that a single seed's final-round number carries non-trivial variance, and a different seed could plausibly shift some of the closer comparisons in Table 4 and Table 5 in either direction, though we consider the largest effects reported here (the CICIoT2023 Gaussian-noise collapse, Section 5.8; Krum's aggregate advantage margin in Table 3) large enough relative to the observed stability gaps to be robust to this concern. We are explicit that this concern is *not* confined to the closer aggregate comparisons: the case study in Section 5.7, our single clearest illustration of ARMOR-FL's mechanism working as intended, itself rests on a last-8-mean margin (0.029) smaller than the round-to-round stability gap the same case study documents for FedAvg (0.397), and we have explicitly flagged it there as a single-seed result requiring multi-seed replication to confirm rather than a settled statistical finding. Repeating the full grid across multiple seeds was outside the compute budget available for this study (Section 4.4 discloses the batch-size compute-budget decision already made for the same reason); we flag single-seed reporting explicitly here rather than presenting point estimates as if they carried the precision of a multi-seed study.

External validity. Our conclusions are grounded in three datasets, three attack types, two malicious-fraction levels, and two non-IID settings; extrapolating beyond this grid – to other attack families (e.g. model-replacement or backdoor attacks, neither of which we test), to malicious fractions above 0.4, to client counts substantially larger or smaller than ten, or to intrusion-detection tasks outside the three network-traffic domains studied here – is not directly supported by our results. The reduced sampling fractions disclosed in Section 4 (10%, 2%, and 1% of the three

datasets respectively, a practical compute-budget decision) mean our results characterize behavior on a substantial but partial slice of each dataset rather than its full extent; we have no specific reason to expect qualitatively different behavior on the withheld data, but we have not verified this directly.

Construct validity. We report final-round and last-8-round-mean accuracy as our primary outcome measures, following the metric convention of the prior work we build on (Section 4.4); accuracy on these three datasets' class distributions (Section 4) may understate performance differences on minority attack classes specifically, since a highly imbalanced multi-class label distribution can allow an aggregator to achieve reasonable overall accuracy while performing poorly on rare, operationally important attack categories. We did not conduct a per-class breakdown in this paper and note it as a natural extension (Section 9).

Conclusion validity. The ablation comparison in Section 5.5 isolates the three corrections of Section 3 as a bundle rather than individually; we cannot, from the data reported here, attribute the observed false-exclusion-rate improvement to any one of the self-shift e-process, the MAD floor, or the threshold-calibrated trust weight in isolation, only to their combination, though the synthetic reproductions in Section 3 do isolate each failure mode (and, correspondingly, each fix) individually on controlled synthetic data. A fully factorial ablation on the real-data grid – eight combinations of the three corrections switched on or off independently, run across all three datasets – would require roughly eight times the compute budget of the single robustness grid reported here and was not feasible within the resources available for this study; we flag it explicitly as the most direct way to strengthen the causal claims in Section 5.5 in future work. We additionally flag an exploratory-versus-confirmatory distinction relevant to the regime identified in Section 6.2 (non-IID data, label-flipping, malicious fraction 0.2): this regime was located by inspecting the twelve-cell breakdown in Table 5 after the full grid completed, not predicted in advance in Section 3 or Section 4, and we did not hold out a separate confirmatory grid slice to test it against. We do not believe this makes the regime spurious – the mechanistic account in Section 3 predicts, independently of having seen these specific numbers, that a stable non-IID baseline and a lighter malicious presence are exactly the conditions favoring the shift mechanism – but we did not pre-register this specific combination before running the grid, and a reader should weigh the finding accordingly, alongside the corroborating attack-type breakdown in Table 4 and the single-seed caveat noted above.

8 Limitations

We state four limitations plainly, since we believe an honest account of what ARMOR-FL does not solve is as important as the mechanism that does work.

First, ARMOR-FL cannot detect a constant attacker: a client that is Byzantine from the very first round and whose bias never changes, provided that bias stays below the gross-outlier threshold, produces no shift signal and is therefore never hard-excluded, only smoothly down-weighted. This is not an implementation gap but a genuine information-theoretic limit – distinguishing a constant, moderate-magnitude attacker from a heterogeneous-but-honest client using only behavioral evidence is not possible in general, since the two are by construction indistinguishable from the aggregator's vantage point. Resolving this would require external information (a trusted validation set, cryptographic attestation of training provenance, or similar) that we do not assume is available.

Second, label-flipping remains genuinely hard to detect via any distance-based signal, ARMOR-FL's included: a client that trains honestly on relabeled data produces a parameter update statistically similar to an honest client's, a limitation ARMOR-FL shares with Krum, trimmed mean, and FoolsGold rather than one specific to our method.

Third, the probation-and-reinstatement mechanism resets a client's accumulated evidence entirely upon reinstatement, giving a previously excluded but still-active attacker a fresh accumulation window – effectively a grace period – before it can be re-excluded. We did not find a way to shorten this window without reintroducing false exclusions for honest clients recovering from a legitimate probation event, and leave a more principled reinstatement rule to future work.

Fourth, ARMOR-FL's own detection precision remains modest (0.167 to 0.312 across the three datasets, Table 6), meaning a non-trivial share of excluded clients over a full run are not actually malicious even after the corrections in Section 3; detection recall is lower still (0.062 to 0.229, Table 6), meaning ARMOR-FL also frequently fails to exclude a truly malicious client at all. We consider this an honest residual cost of the shift-and-population combination rather than evidence the mechanism is not working: the false-exclusion rate under the corrected design is dramatically lower than the uncorrected version's (which reached 62.5% under IID data, Section 3), but it is not zero, and we do not claim otherwise.

Fifth, the ablation comparison in Section 5.5 (Table 8) surfaces an unresolved tension between exclusion aggressiveness and exclusion accuracy: the uncorrected version, which excludes more clients overall, achieves higher detection precision on two of three datasets than the corrected version, which excludes far fewer clients but does not proportionally improve at picking correctly among the exclusions it does make. We do not have a design that improves false-exclusion rate, precision, and recall simultaneously across all three datasets, and we do not believe one is available without additional information beyond per-round behavioral distance – a validation set, cross-client attestation, or a similar external signal (Section 7, construct validity). We view resolving this specific three-way trade-off, rather than any single one of its components in isolation, as the most consequential open problem this paper leaves for future work on the method itself.

9 Conclusion

We set out to build an anytime-valid Byzantine-robust aggregator for federated intrusion detection and found, in the process, that the natural way to do so – comparing each client against the population via a sequential test – carries a structural failure mode that had not, to our knowledge, been previously diagnosed at this level of specificity: any anytime-valid population comparison is guaranteed to eventually exclude a persistently off-center honest client, and the failure is worst precisely when the honest population is most homogeneous, the opposite of the intuitive expectation. ARMOR-FL's self-referential shift test, scale-floored population statistic, and threshold-calibrated trust weight together remove this failure mode, verified both on isolated synthetic reproductions and on three real intrusion-detection benchmarks. The resulting method is not a uniform improvement over the strongest empirical baseline we tested, Krum, but it identifies a specific and reproducible regime – non-IID client populations under lighter label-flipping poisoning – where its formal, cumulative approach to evidence gathering earns a real advantage, together with an anytime-valid statistical guarantee that Krum and FoolsGold do not provide. We release the full implementation, experiment configuration grids, and reassembly-only dataset archives so that this specific failure mode, and the conditions under which our fix helps or does not, can be independently verified and extended.

Several concrete extensions follow directly from the limitations documented above, and we collect them here as a single forward-looking agenda rather than scattering them across sections. Statistically, a fully factorial ablation of the three corrections (Section 7, conclusion validity) and a multi-seed repetition of the full grid (Section 7, internal validity) would sharpen the causal claims this paper can currently only support at the level of the corrections taken together. Mechanistically, the trust-region-style bound on per-round aggregate movement sketched in Section 5.8 is, to our knowledge, untested, and could plausibly close the specific catastrophic-collapse gap that motivated it without requiring Krum's winner-take-all discarding of legitimate contributions. Architecturally, RPCFL's cryptographic clustering guarantee [24] and ARMOR-FL's statistical guarantee address genuinely different threat surfaces (Section 2.2), and a combined system is, as far as we are aware, unexplored. Empirically, a per-class accuracy breakdown (Section 7, construct validity) and an extension of the attack grid to model-replacement and backdoor attacks (Section 7, external validity) would test whether the regime identified in Section 6.2 generalizes beyond the label-flipping-under-non-IID-data case this paper establishes it for. We view none of these as required to make the present contribution's claims valid – each is scoped, in the relevant Threats to Validity paragraph above, as a boundary of what this specific study supports – but together they describe what

we consider the most productive next steps for this line of work.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflict of interest/Competing interests. The author declares no competing interests.

Ethics approval and consent to participate. Not applicable. This work uses publicly available network traffic datasets generated on lab testbeds by their respective creating institutions (CICIDS2017, CICIDS2018 [29]; CICIoT2023 [30]) rather than captured from live production networks, and involves no human subjects or personally identifiable data collected by the author.

Dual-use considerations. The accompanying release includes configuration files that reproduce specific poisoning attacks (label-flipping, Gaussian-noise, and the adaptive ALIE attack) against a real federated intrusion-detection backbone. We judge the marginal risk of this release to be low: all three attacks are reimplementations of already-published techniques [48] rather than novel attack methods, the datasets are lab-testbed traffic rather than live production captures, and reproducing the attack grid requires no credentials, access, or target model beyond what the release itself provides in a fully offline simulation. We disclose this consideration explicitly rather than silently, consistent with the reporting stance taken throughout this paper.

Consent for publication. Not applicable.

Data availability. The datasets analyzed (CICIDS2017, CICIDS2018, CICIoT2023) are publicly available from the Canadian Institute for Cybersecurity and are redistributed in the accompanying code repository as checksummed archive chunks for reassembly, avoiding the need to re-download from the original hosts.

Materials availability. Not applicable.

Code availability. The full ARMOR-FL implementation, experiment configuration files, and analysis scripts are available in the accompanying code repository.

Author contribution. Q.-V.D. conceived the study, designed and implemented ARMOR-FL, ran all experiments, performed the analysis, and wrote the manuscript.

Appendix A Full per-combination results

Tables A1, A2, and A3 report final-round accuracy for every individual attack-type × malicious-fraction × non-IID-α combination in the robustness grid, the complete disaggregation underlying the marginal summaries in Tables 3, 4, and 5. We include these in full so that any specific comparison discussed in the main text can be checked directly against its source cell, and so that a reader interested in a combination we did not single out for discussion can locate it exactly.

Several individual cells in Tables A1–A3 repeat an identical value across multiple aggregators for the same combination (for instance, Gaussian-noise on CICIDS2017 shows 0.803 for ARMOR-FL, FedAvg, and FoolsGold across every malicious-fraction and α combination). We confirmed this is not a data-processing artifact but reflects the underlying attack's actual effect at the noise magnitude and malicious fractions we test: on CICIDS2017 specifically, the additive perturbation is not disruptive enough, at 20% or 40% malicious clients, to move the aggregate final-round accuracy away from what the backbone classifier converges to regardless of which of the three averaging-based rules is used, consistent with the discussion in Section 5.2. Krum, which discards rather than averages, is the only aggregator whose accuracy on these rows differs from the other three, which is itself informative: the recurring identical values are a property of the averaging-based rules under this specific attack-dataset combination, not evidence that all four aggregators behave identically.

Table A1 CICIDS2017: final-round accuracy, every attack-fraction-α combination

Attack	Mal. frac.	α	ARMOR-FL	FedAvg	FoolsGold	Krum
ALIE	0.2	0.5	0.874	0.804	0.803	0.809
ALIE	0.2	IID	0.876	0.904	0.756	0.851
ALIE	0.4	0.5	0.803	0.803	0.803	0.803
ALIE	0.4	IID	0.208	0.652	0.803	0.803
Gaussian-noise	0.2	0.5	0.803	0.803	0.803	0.656
Gaussian-noise	0.2	IID	0.803	0.803	0.803	0.738
Gaussian-noise	0.4	0.5	0.803	0.803	0.803	0.830
Gaussian-noise	0.4	IID	0.803	0.803	0.803	0.803
Label-flip	0.2	0.5	**0.891**	0.799	0.735	0.811
Label-flip	0.2	IID	0.864	0.825	0.846	**0.904**
Label-flip	0.4	0.5	0.065	0.485	0.064	**0.874**
Label-flip	0.4	IID	0.547	0.233	0.552	**0.838**

Table A2 CICIDS2018: final-round accuracy, every attack-fraction-α combination

Attack	Mal. frac.	α	ARMOR-FL	FedAvg	FoolsGold	Krum
ALIE	0.2	0.5	0.841	0.831	0.831	**0.857**
ALIE	0.2	IID	0.770	0.804	0.831	**0.968**
ALIE	0.4	0.5	0.831	0.831	0.831	0.831
ALIE	0.4	IID	**0.840**	0.831	0.831	0.831
Gaussian-noise	0.2	0.5	0.831	0.831	0.831	**0.886**
Gaussian-noise	0.2	IID	0.831	0.831	0.831	**0.917**
Gaussian-noise	0.4	0.5	0.831	0.831	0.831	0.831
Gaussian-noise	0.4	IID	0.831	0.831	0.831	**0.940**
Label-flip	0.2	0.5	**0.831**	0.759	0.687	0.763
Label-flip	0.2	IID	0.871	**0.945**	0.397	0.936
Label-flip	0.4	0.5	0.568	**0.831**	0.092	0.050
Label-flip	0.4	IID	0.648	0.745	0.792	**0.914**

Table A3 CICIoT2023: final-round accuracy, every attack-fraction-α combination

Attack	Mal. frac.	α	ARMOR-FL	FedAvg	FoolsGold	Krum
ALIE	0.2	0.5	0.813	**0.819**	0.723	0.599
ALIE	0.2	IID	**0.832**	0.757	0.764	0.822
ALIE	0.4	0.5	0.723	0.723	0.707	0.723
ALIE	0.4	IID	0.023	**0.770**	0.023	0.011
Gaussian-noise	0.2	0.5	0.023	0.023	0.023	**0.707**
Gaussian-noise	0.2	IID	0.023	0.023	0.023	**0.822**
Gaussian-noise	0.4	0.5	0.023	0.023	0.023	**0.595**
Gaussian-noise	0.4	IID	0.023	0.023	0.023	**0.823**
Label-flip	0.2	0.5	0.792	0.803	**0.802**[1]	0.694
Label-flip	0.2	IID	**0.828**	0.651	0.824	0.725
Label-flip	0.4	0.5	0.483	0.157	0.157	**0.706**
Label-flip	0.4	IID	**0.488**	0.185	0.443	0.193

Appendix B Full per-combination detection statistics

Tables B4, B5, and B6 disaggregate ARMOR-FL's own detection precision, detection recall, and benign false-exclusion rate (Section 4.5) by individual combination, underlying the marginal summaries in Tables 6 and 7. A precision cell marked "–" indicates ARMOR-FL excluded no client at all during that run, making precision (a ratio over excluded clients) undefined rather than zero; we report it as undefined rather than substituting an arbitrary default, since treating an undefined ratio as zero would misleadingly imply a failed detection attempt rather than the absence of any exclusion event.

Recall is zero on a majority of individual combinations across all three datasets, including several where the benign false-exclusion rate is simultaneously non-zero (e.g. CICIDS2018 ALIE at malicious fraction 0.4, both α settings: recall 0.000, benign FER 0.333–0.500) – ARMOR-FL excluded only honest clients and no malicious ones at all in these specific runs. This is the most granular evidence in this paper for the honest limitation stated in Section 8: detection recall is not merely lower than precision in aggregate (Table 6), it is frequently exactly zero at the level of an individual run, meaning ARMOR-FL's population-and-shift combination, at the significance levels and betting fractions of Table 2, often accumulates the twenty-five rounds of our grid without ever crossing the exclusion threshold against a truly malicious client, even while it does, in the same run, occasionally cross that threshold against an honest one.

Appendix C Reproducibility checklist

We report the following details explicitly to support independent reproduction, following standard reproducibility-checklist practice for empirical machine-learning papers. **Hardware.** All experiments were run on a single workstation with one NVIDIA GPU; no multi-GPU or distributed training was used, and federated clients are simulated sequentially within a single process rather than as physically separate machines. **Software.** Python 3.13 (conda environment), with the CUDA lazy-module-loading behavior explicitly disabled (`CUDA_MODULE_LOADING=EAGER`) to work around a Windows-specific crash we encountered during development. **Randomness.** Each of the 144 grid cells (three datasets $\times$ four aggregators $\times$ three attacks $\times$ two malicious fractions $\times$ two α settings, minus the cells FedAvg/Krum/FoolsGold do not require an attack-free run for) uses a single fixed random seed per configuration, not averaged over multiple seeds (Section 7, internal validity). **Compute budget.** The full robustness grid (144 runs $\times$ 25 rounds) completes in approximately 13 hours of wall-clock time at the batch size disclosed in Section 4.4, following the 200-plus-hour infeasibility of the batch-size-32 configuration that motivated that disclosed deviation. **Configuration files.** Every result in this paper corresponds to a named YAML configuration file in the accompanying repository (`configs/*_robustness.yaml` and `configs/*_comparability.yaml`), and every reported number can be regenerated by re-running the corresponding configuration through the release pipeline without modification. **Data.** The three datasets are redistributed as checksummed archive chunks for reassembly rather than raw re-hosted copies, per the Data Availability declaration below, to avoid both re-download friction and any redistribution restriction on the original hosts' full raw files.

Appendix D Worked numerical example

To make Algorithm 1 concrete beyond its symbolic description, we work a single simplified round by hand for a toy federation of four active clients (rather than the six of ten used in the real grid, for brevity), of which client 4 is malicious.

Suppose the four clients' distances to the round's trimmed-mean reference point are $d = (1.0, 1.1, 0.9, 4.5)$ – clients 1–3 honest and closely clustered, client 4 an outlier. The population scale estimate is $\mathrm{MAD}^{(t)} = \mathrm{median}(d) = 1.05$; suppose this exceeds the floor $0.5 \cdot \mathrm{median}(\text{recent raw MAD history})$, so no flooring correction applies this round and $\mathrm{MAD}^{(t)}_{\mathrm{eff}} = 1.05$.

Table B4 CICIDS2017: ARMOR-FL detection statistics, every attack-fraction-α combination

Attack	Mal. frac.	α	Precision	Recall	Benign FER
ALIE	0.2	0.5	0.667	1.000	0.125
ALIE	0.2	IID	–	0.000	0.000
ALIE	0.4	0.5	0.000	0.000	0.500
ALIE	0.4	IID	0.000	0.000	0.167
Gaussian-noise	0.2	0.5	0.000	0.000	0.125
Gaussian-noise	0.2	IID	1.000	1.000	0.000
Gaussian-noise	0.4	0.5	–	0.000	0.000
Gaussian-noise	0.4	IID	–	0.000	0.000
Label-flip	0.2	0.5	0.500	0.500	0.125
Label-flip	0.2	IID	0.000	0.000	0.125
Label-flip	0.4	0.5	0.333	0.250	0.333
Label-flip	0.4	IID	–	0.000	0.000

Table B5 CICIDS2018: ARMOR-FL detection statistics, every attack-fraction-α combination

Attack	Mal. frac.	α	Precision	Recall	Benign FER
ALIE	0.2	0.5	0.000	0.000	0.250
ALIE	0.2	IID	–	0.000	0.000
ALIE	0.4	0.5	0.000	0.000	0.333
ALIE	0.4	IID	0.000	0.000	0.500
Gaussian-noise	0.2	0.5	–	0.000	0.000
Gaussian-noise	0.2	IID	1.000	0.500	0.000
Gaussian-noise	0.4	0.5	–	0.000	0.000
Gaussian-noise	0.4	IID	–	0.000	0.000
Label-flip	0.2	0.5	0.000	0.000	0.250
Label-flip	0.2	IID	0.000	0.000	0.125
Label-flip	0.4	0.5	0.333	0.250	0.333
Label-flip	0.4	IID	0.000	0.000	0.167

Table B6 CICIoT2023: ARMOR-FL detection statistics, every attack-fraction-α combination

Attack	Mal. frac.	α	Precision	Recall	Benign FER
ALIE	0.2	0.5	0.500	1.000	0.250
ALIE	0.2	IID	–	0.000	0.000
ALIE	0.4	0.5	0.000	0.000	0.167
ALIE	0.4	IID	0.000	0.000	0.500
Gaussian-noise	0.2	0.5	0.000	0.000	0.250
Gaussian-noise	0.2	IID	–	0.000	0.000
Gaussian-noise	0.4	0.5	–	0.000	0.000
Gaussian-noise	0.4	IID	–	0.000	0.000
Label-flip	0.2	0.5	0.000	0.000	0.375
Label-flip	0.2	IID	–	0.000	0.000
Label-flip	0.4	0.5	0.667	0.500	0.167
Label-flip	0.4	IID	–	0.000	0.000

The resulting population-deviation statistics are $z = (0.952, 1.048, 0.857, 4.286)$.

Each client's e-process multiplier is $e_t = 1 + 0.5(z - 1)$ (Section 3.2), giving $e = (0.976, 1.024, 0.929, 2.643)$. Suppose each client's population e-process entered this round at $E_{t-1} = (1.2, 0.9, 1.1, 8.5)$ (accumulated from prior rounds); the updated values are $E_t = (1.171, 0.922, 1.022, 22.474)$. With $\alpha_{\text{pop}} = 0.05$, the exclusion threshold is $1/\alpha_{\text{pop}} = 20$; only client 4 exceeds it ($22.474 \geq 20$), so clients 1–3 continue unaffected by the population gate.

Client 4's shift statistic (Section 3.3) compares its current $z = 4.286$ to the median of its own two-round baseline; suppose that baseline was $(0.9, 1.0)$ (client 4 looked ordinary before an attack began at this round), giving shift $= 4.286/0.95 = 4.512$, comfortably exceeding the shift threshold. Since client 4 satisfies both the population-evidence condition and the shift condition, it is hard-excluded this round: $w_4^{(t)} = 0$, and it enters probation.

For the surviving clients, the trust weight (Section 3.4) uses $\tau = -\log(0.05) \approx 2.996$. Client 1's population e-process is $E_1 = 1.171$, giving $\log E_1 = 0.158$ and $\text{trust\_scale}_1 = \text{sigmoid}(-4(0.158/2.996 - 1)) = \text{sigmoid}(3.789) \approx 0.978$: client 1 retains essentially full weight, appropriately, since its deviation is negligible relative to the exclusion threshold. Had the naive, unrescaled sigmoid been used instead ($\text{sigmoid}(-4 \times 0.158) = \text{sigmoid}(-0.632) \approx 0.347$), client 1 – an entirely honest client with only mild deviation – would have been reduced to roughly a third of its due weight, the precise failure mode described in Section 3.4 and confirmed on real data via the trust-weight trace that motivated that correction.

We now contrast this with the undetectable case discussed as the first limitation in Section 8: a constant, moderate-magnitude attacker present from round one. Suppose client 4 is instead Byzantine from the start, with $z_4^{(t)} \approx 2.0$ (below the gross-outlier threshold of 4) on every round, including the two-round clean-baseline period (Section 4.3). Its shift baseline, computed from its own first two rounds, is then $\text{median}(2.0, 2.0) = 2.0$, and its shift statistic in every subsequent round is $\text{shift}_4^{(t)} = 2.0/2.0 = 1.0$ – indistinguishable from a client that has never deviated at all, since the baseline itself already encodes the attacker's bias. The population e-process for this client still accumulates evidence steadily (by Proposition 2, since $\mu = 2.0 > 1$ persistently) and will eventually cross the population threshold alone, but the shift gate of Section 3.3 never fires, so hard exclusion never triggers under the AND condition in Algorithm 1; only the smooth trust-weight downweighting of Section 3.4 applies, converging to a partial but non-zero weight as $\log E_{\text{pop},4}^{(t)}$ grows past τ. This is the precise mechanical realization of the information-theoretic limit stated in Section 8: no adjustment to b, α_{shift}, or the gross-outlier threshold recovers hard exclusion here, because the client's own baseline is contaminated by the same bias the test is trying to detect, and no signal in this paper's threat model (Section 3.1) distinguishes a contaminated baseline from a genuinely elevated but honest one.

Appendix E Notation summary

Table E7 collects the symbols introduced across Sections 3.1–3 for reference, since the method combines notation from anytime-valid testing, robust statistics, and standard federated learning.

Appendix F Baseline aggregation pseudocode

For direct comparison with Algorithm 1, Algorithms 2 and 3 give Krum and FoolsGold in the same notation, corresponding to the descriptions in Section 3.3.

Algorithm 2 Krum aggregation, one round

Input: active client updates $\{\theta_k^{(t)}\}_{k \in \mathcal{S}_t}$, assumed Byzantine count f
for each active client k **do**
    $D_k \leftarrow$ sum of squared distances from $\theta_k^{(t)}$ to its $|\mathcal{S}_t| - f - 2$ nearest neighbors in $\{\theta_j^{(t)}\}_{j \in \mathcal{S}_t}$
end for
$k^* \leftarrow \arg\min_k D_k$
Output: $\theta^{(t+1)} \leftarrow \theta_{k^*}^{(t)}$

Table E7 Notation summary

Symbol	Meaning
K	Number of clients in the federation
$\mathcal{M}$	Unknown subset of malicious clients
$\mathcal{S}_t$	Clients active (selected) in round t
$\theta^{(t)}$	Global parameter vector after round t
$\theta_k^{(t)}$	Client k's local update in round t
$(\mathcal{F}_t)$	Filtration of aggregator information through round t
(E_t)	An e-process (Definition 1)
α	Significance level in Ville's inequality (Theorem 1)
λ	Betting fraction in the testing-by-betting construction
μ_0	Fixed null mean for the population statistic ($\mu_0 = 1$)
$m^{(t)}$	Coordinate-wise trimmed-mean reference point, round t
$d_k^{(t)}$	Client k's distance to $m^{(t)}$
$\mathrm{MAD}^{(t)}$	Round-t median-absolute-deviation scale estimate
$\mathrm{MAD}_{\mathrm{eff}}^{(t)}$	Floored (corrected) scale estimate
$z_k^{(t)}$	Client k's population-deviation statistic, $d_k^{(t)}/\mathrm{MAD}_{\mathrm{eff}}^{(t)}$
$E_{\mathrm{pop},k}^{(t)}$	Client k's population e-process value
$\mathrm{shift}_k^{(t)}$	Client k's self-shift statistic (Section 3.3)
b	Shift-test baseline window length ($b = 2$)
$\alpha_{\mathrm{pop}}, \alpha_{\mathrm{drift}}, \alpha_{\mathrm{shift}}$	Per-test significance levels (each 0.05)
τ	Trust-weight rescaling threshold, $-\log(\alpha_{\mathrm{pop}})$
β	Trust-weight sigmoid sharpness ($\beta = 4$)
$w_k^{(t)}$	Client k's aggregation weight, round t
n_k	Client k's local sample count

Algorithm 3 FoolsGold aggregation, one round

Input: active client updates $\{\theta_k^{(t)}\}_{k\in\mathcal{S}_t}$, per-client update history
for each pair of active clients (i, j) **do**
    $c_{ij} \leftarrow$ cosine similarity between i's and j's cumulative update history
end for
for each active client k **do**
    $v_k \leftarrow \max_{j\neq k} c_{kj}$ (highest similarity to any other client)
    $w_k \leftarrow 1 - v_k$, rescaled and logit-transformed per [3]
end for
Output: $\theta^{(t+1)} \leftarrow$ weighted average of $\{\theta_k^{(t)}\}$ with weights $\{w_k\}$

Comparing Algorithm 2 against Algorithm 1 makes concrete the mechanical explanation offered in Section 6.1: Krum's output is a single client's update, verbatim, with no blending step at all, whereas ARMOR-FL (and FoolsGold, Algorithm 3) always produce a weighted combination across every active client passing the exclusion gate. The absence of any blending in Krum is precisely why a single successfully-identified honest client fully determines the round's aggregate regardless of how many other active clients are compromised, at the cost of discarding every other client's legitimate contribution outright even when most of them are honest.

References

[1] McMahan, B., Moore, E., Ramage, D., Hampson, S., Agüera y Arcas, B.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS 2017). Proceedings of Machine Learning Research, vol. 54, pp. 1273–1282 (2017)

[2] Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems 30 (NeurIPS 2017) (2017)

[3] Fung, C., Yoon, C.J.M., Beschastnikh, I.: The limitations of federated learning in Sybil settings. In: 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020) (2020)

[4] Waudby-Smith, I., Ramdas, A.: Estimating means of bounded random variables by betting. Journal of the Royal Statistical Society Series B: Statistical Methodology **86**(1), 1–27 (2024) https://doi.org/10.1093/jrsssb/qkad009

[5] El Mhamdi, E.M., Guerraoui, R., Rouault, S.: The hidden vulnerability of distributed learning in Byzantium. In: Proceedings of the 35th International Conference on Machine Learning (ICML 2018). Proceedings of Machine Learning Research, vol. 80 (2018)

[6] Shejwalkar, V., Houmansadr, A.: Manipulating the Byzantine: optimizing model poisoning attacks and defenses for federated learning. In: Proceedings of the 2021 Network and Distributed System Security Symposium (NDSS 2021) (2021). https://doi.org/10.14722/ndss.2021.24498

[7] Karimireddy, S.P., He, L., Jaggi, M.: Learning from history for Byzantine robust optimization. In: Proceedings of the 38th International Conference on Machine Learning (ICML 2021). Proceedings of Machine Learning Research, vol. 139 (2021)

[8] Yang, R., He, H., Wang, Y., Qu, Y., Zhang, W.: Dependable federated learning for IoT intrusion detection against poisoning attacks. Computers & Security **132**, 103381 (2023) https://doi.org/10.1016/j.cose.2023.103381

[9] Lavaur, L., Busnel, Y., Autrel, F.: Investigating the impact of label-flipping attacks against federated learning for collaborative intrusion detection systems. Computers & Security **150**, 104462 (2025) https://doi.org/10.1016/j.cose.2025.104462

[10] Rezaei, H., Taheri, R., Shojafar, M., Foh, C.H.: GRAF-IDS: graph-based clustering as aggregation for federated intrusion detection systems in IoT network. Neural Computing and Applications **37**, 18401–18423 (2025) https://doi.org/10.1007/s00521-025-11385-1

[11] Rezaei, H., Taheri, R., Jordanov, I., Shojafar, M.: Federated RNN for intrusion detection systems in IoT environment under adversarial attack. Journal of Network and Systems Management **33**, 82 (2025) https://doi.org/10.1007/s10922-025-09963-8

[12] Sameera, K.M., Vinod, P., Rocha, A., Rehiman, K.A.R., Conti, M.: WeiDetect: Weibull distribution-based defense against poisoning attacks in federated learning for network intrusion detection systems. Journal of Information Security and Applications **95**, 104275 (2025) https://doi.org/10.1016/j.jisa.2025.104275

[13] Zukaib, U., Cui, X., Niaz, F., Liang, D., Zheng, C., Din, S.U.: Defending federated learning-based intrusion detection systems against model poisoning attacks. Neurocomputing (2026) https://doi.org/10.1016/j.neucom.2026.134208

[14] Latif, N., Ma, W., Ahmad, H.B.: FedDBC: density-based defense against collusion attacks in federated intrusion detection for IoT. Computer Networks (2026) https://doi.org/10.1016/j.comnet.2026.112497

[15] Chowdhury, A.P., Nur, F.N., Islam, A., Alam, K., Karim, A., Shah, M.A.: FLEX-IDS: secure, explainable federated intrusion detection system for edge-of-things under adversarial conditions. Computers and Electrical Engineering (2026) https://doi.org/10.1016/j.compeleceng.2025.110827

[16] Akter, R., Naizheng, B., Ullah, I., Singh, S., Singh, A., Iqbal, M.: Fair client selection and encrypted aggregation: federated learning framework for intrusion detection in

resource-constrained networks. Cluster Computing **29**, 164 (2026) https://doi.org/10.1007/s10586-025-05905-w

[17] Buyuktanir, B., Altinkaya, Ş., Karatas Baydogmus, G., Yildiz, K.: Federated learning in intrusion detection: advancements, applications, and future directions. Cluster Computing **28**(7), 473 (2025) https://doi.org/10.1007/s10586-025-05325-w

[18] Mohawesh, R., Al-Obiedollah, H., Maqsood, S., Bany Salameh, H.: Lightweight large language model based hierarchical federated learning for B5G enabled IoT intrusion detection networks. Cluster Computing **29**(3), 195 (2026) https://doi.org/10.1007/s10586-026-05939-8

[19] Bella, K., Guezzaz, A., Ravi, V., Benkirane, S., Mohy-eddine, M., Azrour, M., Ennajar, S.: Enhancing data privacy in cyber-physical systems with federated learning-based intrusion detection. Cluster Computing **29**(6), 346 (2026) https://doi.org/10.1007/s10586-026-06001-3

[20] Godavarthi, D., Mahesh, T.U., Jithendar, Mohanty, S.N., Dash, J.K.: Improving IoT security through federated deep Q-learning with realistic traffic modelling. Cluster Computing **29**(6), 399 (2026) https://doi.org/10.1007/s10586-026-06160-3

[21] Houichi, M., Jaidi, F., Bouhoula, A.: Trust-aware federated intrusion detection system framework for privacy-preserving smart city IoT. Computer Networks (2026) https://doi.org/10.1016/j.comnet.2026.112616

[22] Ram, A., Chakraborty, S.K.: Trust-aware blockchain-assisted framework for federated intrusion detection in SDNs. Computer Networks (2026) https://doi.org/10.1016/j.comnet.2026.112360

[23] Dang, Q.-V., Vo, T.-B.-D., Nguyen, N.-S.-A.: FORTRESS-FL: Byzantine-robust and privacy-preserving federated orchestration for next-generation networks. Array **29**, 100680 (2026) https://doi.org/10.1016/j.array.2026.100680

[24] Chen, P., Tan, W., Zhong, Y., Wang, H., Fan, L., Weng, J.: RPCFL: a byzantine-robust and privacy-preserving clustered federated learning framework. Cluster Computing **29**(2), 125 (2026) https://doi.org/10.1007/s10586-025-05900-1

[25] Dang, Q.-V., Le, D., Dinh, M.N., Nguyen, N.-S.-A., Vo, T.-B.-D.: ST-FedXIDS: a spatiotemporal federated explainable intrusion detection system with drift-adaptive graph learning for high-density public WiFi environments. International Journal of Advances in Signal and Image Sciences **12**(1), 873–886 (2026) https://doi.org/10.29284/zcrk2p65

[26] Dang, Q.-V.: Utilizing uncertainty measures to improve the performance of intrusion detection systems. SN Computer Science **7**(4), 344 (2026) https://doi.org/10.1007/s42979-026-04923-8

[27] Ramdas, A., Grünwald, P., Vovk, V., Shafer, G.: Game-theoretic statistics and safe anytime-valid inference. Statistical Science (2023) https://doi.org/10.1214/23-sts894

[28] Howard, S.R., Ramdas, A., McAuliffe, J., Sekhon, J.: Time-uniform, nonparametric, nonasymptotic confidence sequences. The Annals of Statistics (2021) https://doi.org/10.1214/20-aos1991

[29] Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018), pp. 108–116 (2018). https://doi.org/10.5220/0006639801080116

[30] Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A.: CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors **23**(13), 5941 (2023) https://doi.org/10.3390/s23135941

[31] Wahab, S.A., Sultana, S., Tariq, N., Mujahid, M., Khan, J.A., Mylonas, A.: Multi-class intrusion detection for DDoS attacks in IoT

networks using deep learning and transformers. Sensors **25**(15), 4845 (2025) https://doi.org/10.3390/s25154845

[32] Wakili, A., Bakkali, S.: Resilient IoT intrusion detection system using hybrid feature selection and explainable ensemble learning. Results in Engineering (2025) https://doi.org/10.1016/j.rineng.2025.107392

[33] Zhou, Q., Mao, X., Chen, Y.: FedSE-1DSqueezeNet: a lightweight federated intrusion detection system for Internet of Things. Cluster Computing **29**(11) (2026) https://doi.org/10.1007/s10586-026-06482-2

[34] Xu, C., Li, D., Liu, Z., Yang, J., Shen, Q., Tong, N.: Few-shot network intrusion detection method based on multi-domain fusion and cross-attention. PLOS ONE **20**, 0327161 (2025) https://doi.org/10.1371/journal.pone.0327161

[35] Zhang, C., Li, J., Wang, N., Zhang, D.: Intrusion detection method based on transformer and CNN-BiLSTM in IoT. Sensors **25**(9), 2725 (2025) https://doi.org/10.3390/s25092725

[36] Palani, S., S, P., Durairaj, P., Praveen S, L., Victor, S.: Comparative study of transformer-based architecture and CNN-LSTM hybrid model for network intrusion detection using CSE-CIC-IDS 2018 dataset. Turkish Journal of Engineering **10**(3) (2026) https://doi.org/10.31127/tuje.1843251

[37] Xue, Y., Kang, C., Yu, H.: HAE-HRL: a novel autoencoder and hybrid enhanced LSTM-CNN residual network for network intrusion detection. Computers & Security **151**, 104328 (2025) https://doi.org/10.1016/j.cose.2025.104328

[38] Sireesha, D., Kumar, K.A.: Lightweight intrusion detection system using multi-scale attention 1D CNN for large-scale IoT. Frontiers in Artificial Intelligence **9**, 1857306 (2026) https://doi.org/10.3389/frai.2026.1857306

[39] Zhao, X., Fok, K.W., Thing, V.L.L.: Enhancing network intrusion detection performance using generative adversarial networks. Computers & Security (2024) https://doi.org/10.1016/j.cose.2024.104005

[40] Dang, Q.-V., Pham, T.-H.: Detecting IoT malware using federated learning. In: Data Science and Applications. Lecture Notes in Networks and Systems, pp. 73–83. Springer, ??? (2024). https://doi.org/10.1007/978-981-99-7862-5_6

[41] Dang, Q.-V., Pham, T.-H.: Kernel methods for conformal prediction to detect botnets. In: Artificial Intelligence: Theory and Applications. Lecture Notes in Networks and Systems, pp. 29–41. Springer, ??? (2024). https://doi.org/10.1007/978-981-99-8476-3_3

[42] Dang, Q.-V.: Learning to transfer knowledge between datasets to enhance intrusion detection systems. In: Computational Intelligence. Lecture Notes in Electrical Engineering, pp. 39–46. Springer, ??? (2023). https://doi.org/10.1007/978-981-19-7346-8_4

[43] Dang, Q.-V.: Intrusion detection in Internet of Things environment. In: Advances in Digital Science – ADS 2022, pp. 26–34. Institute of Certified Specialists (ICS), ??? (2022). https://doi.org/10.33847/978-5-6048575-0-2_2

[44] Dang, Q.-V.: Using machine learning for intrusion detection systems. Computing and Informatics **41**(1), 12–33 (2022) https://doi.org/10.31577/cai_2022_1_12

[45] Dang, Q.-V.: Improving the performance of the intrusion detection systems by the machine learning explainability. International Journal of Web Information Systems **17**(5), 537–555 (2021) https://doi.org/10.1108/ijwis-03-2021-0022

[46] Thein, T.T., Shiraishi, Y., Morii, M.: Personalized federated learning-based intrusion detection system: poisoning attack and defense. Future Generation Computer Systems **153**, 182–192 (2024) https://doi.org/10.1016/j.future.2023.10.005

[47] Wald, A.: Sequential tests of statistical hypotheses. The Annals of Mathematical Statistics **16**(2), 117–186 (1945) https://doi.org/10.1214/aoms/1177731118

[48] Baruch, G., Baruch, M., Goldberg, Y.: A little is enough: circumventing defenses for distributed learning (2019)